DOC-ID : ACC-DEL-001

Adaptive Cruise Control A-SPICE 산출물 패키지

1/10 스케일 RC카 — 시스템 / 소프트웨어 / 안전 산출물

프로젝트 정보

프로젝트명	1/10 스케일 RC카 기반 Adaptive Cruise Control 시스템
적용 표준	Automotive SPICE 4.0 + ISO 26262:2018
HW 플랫폼	NXP TRK-MPC5606B · Arduino Uno × 2 · Raspberry Pi 5
개발 기간	1개월 (W1 컨셉 → W2 설계 → W3 구현 → W4 자격·시연)
팀	**3조 개포동 잡초** (새싹 SeSAC · 5인)

팀 구성 — 역할 분담

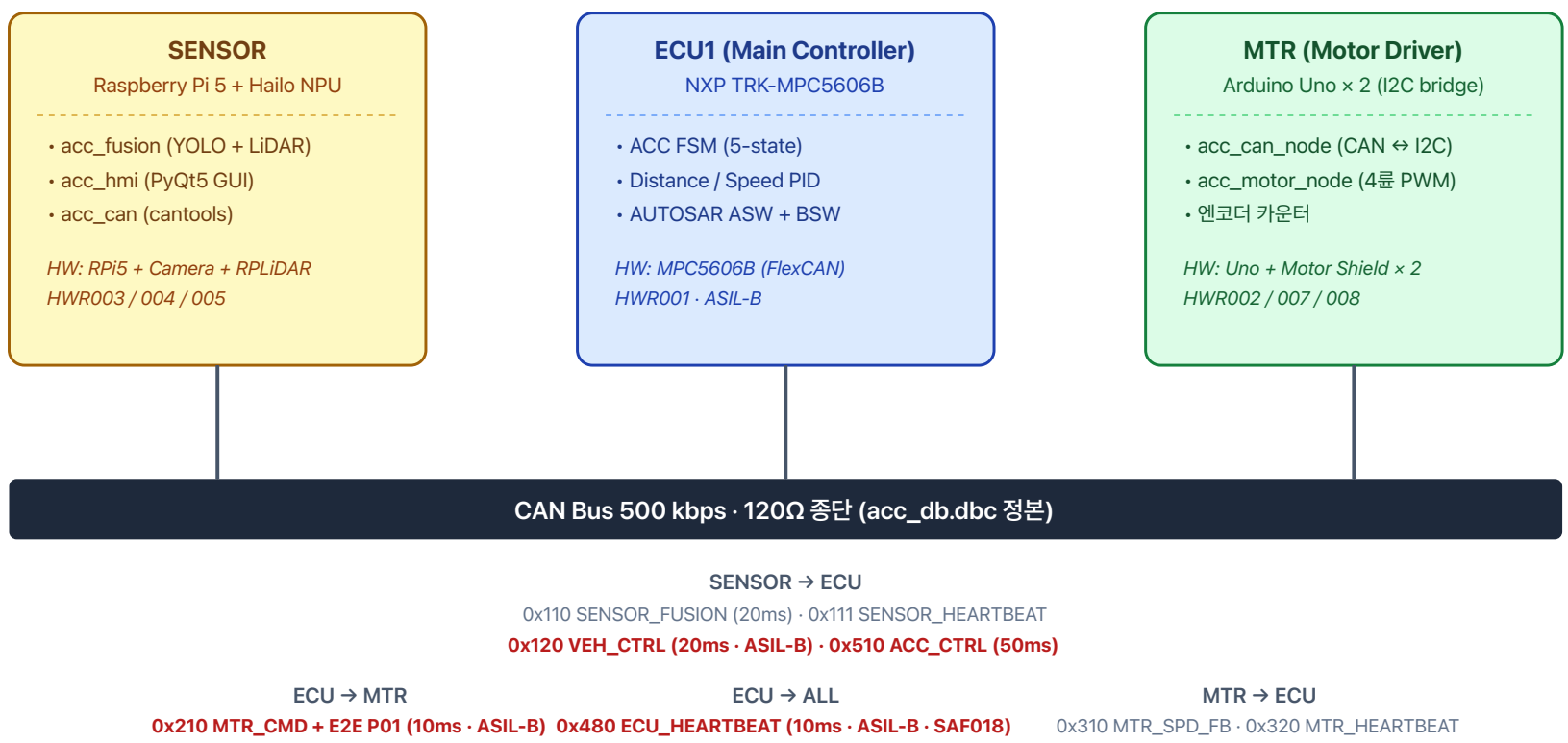
역할	담당	주요 책임
PM · ALL-ROUND	**박종범**	전 리포 setup · DBC v1~v3 (E2E P01) · AUTOSAR Composition · CanCommunication · MotorControl · MBD 인프라 · RPi 초기 setup
RPi · HMI	**김유림**	RPi HMI 메인 (PyQt5 dashboard / 페달 / 카메라뷰) · acc_can DBC-driven 리팩터 (cantools) · MTR_SPD_FB RX · AUTOSAR HmiManager + Diagnostics SWC
FSM · Stateflow	**배정우**	AccStateMachine SWC 전담 · Stateflow FSM 5-state 구현 · SWR / SYS 정합성 스윕 (v0.3 → v0.5.3) · 요구사항 갭 분석 (SYS001 / 029 / 033 / 034)
Arduino · MCU	**유다희**	Arduino 모터 노드 전담 · 시스템 구조 변경 (앞/뒤 → CAN/MOTOR 분리) · CAN 피드백 안정성 fix · AUTOSAR HmiManager v0.3 단순화
mobilgene · PID · CANDB	**엄지혜**	**mobilgene MPC5606B 전담** (BSW/RTE/EcuExtract 생성, OS Task 매핑, Data Mappings 자동화) · PID 제어 튜닝 · 초기 CAN 설정 · CANDB 정합성 검토 · ARXML import / Composition 빌드

📐 시스템 개요 다이어그램

ACC 프로젝트 전체를 한눈에 보기 위한 6개 핵심 다이어그램. 각 다이어그램은 A4 한 페이지에 한 개씩 배치되며, 각 프로세스 챕터에서도 관련 다이어그램이 재등장합니다.

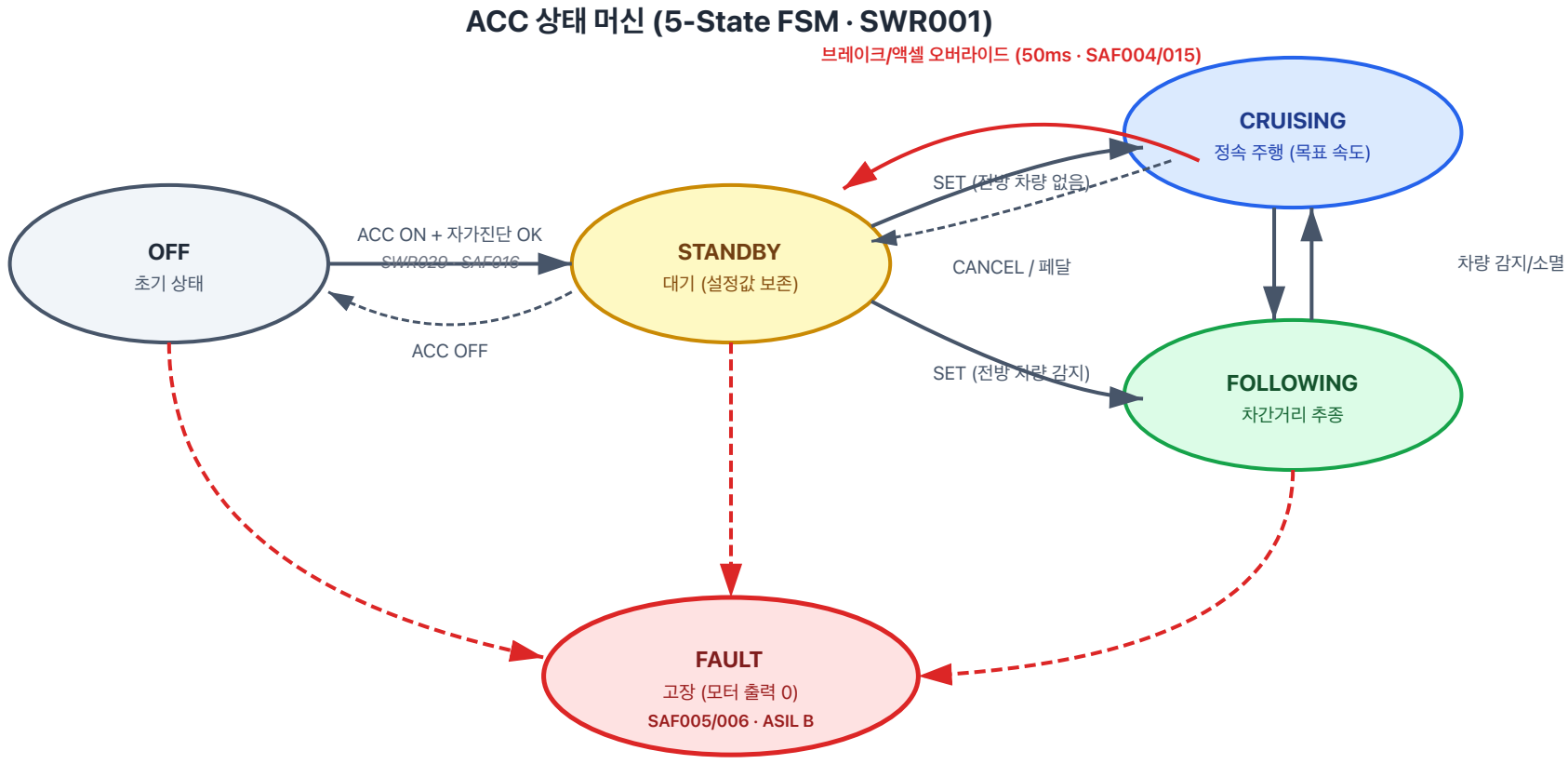
① 시스템 아키텍처 — 3-노드 CAN 토폴로지

ACC 시스템 아키텍처 (3-노드 CAN 토폴로지 · 500 kbps)



② ACC 상태 머신 — 5-state FSM

ACC 상태 머신 (5-State FSM · SWR001)



전이 트리거 범례

정상 전이 (SET/RES, 차량 감지) · FAULT 전이 (센서/CAN 고장 · SAF005/006) · 오버라이드 (브레이크/액셀 50ms 응답)

근거: STK009 · SYS001~005 · SWR001~003 · SAF004 (ASIL B 응답시간 50ms)

③ V-모델 — SYS / SWE 매핑 + ISO 26262 체인

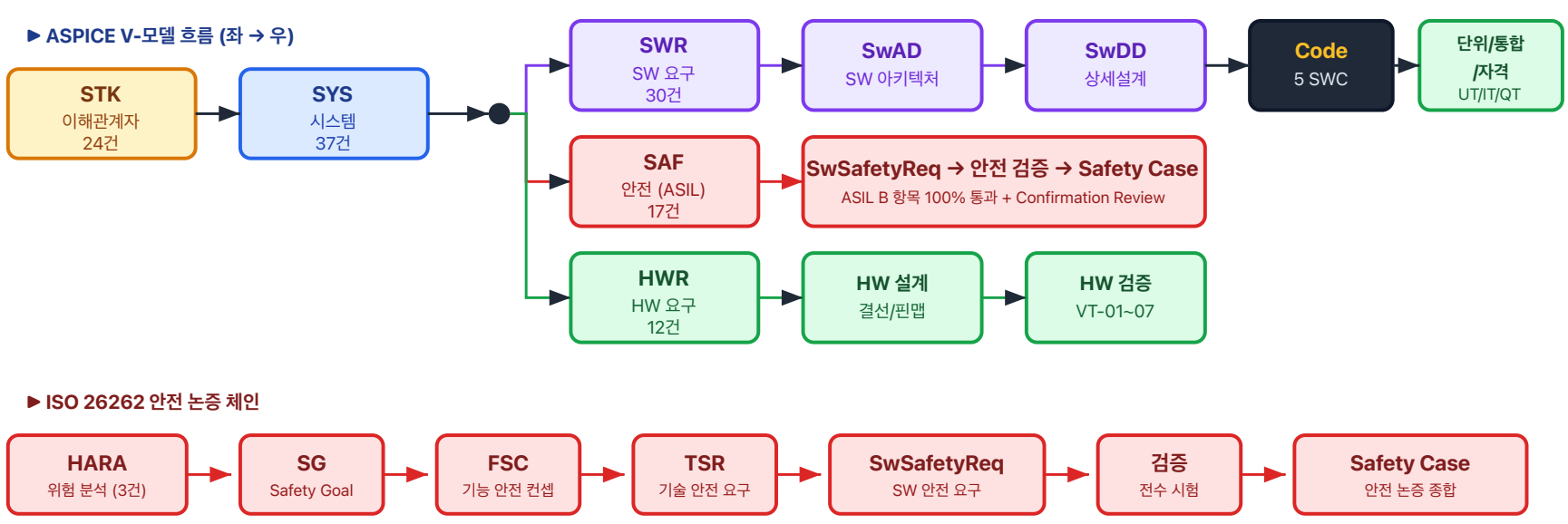
V-모델 — ASPICE 4.0 SYS/SWE 매핑 + ISO 26262 안전 체인

설계 (좌) · 검증 (우) — 양방향 추적성 점선 = 같은 레벨 끼리 검증 매핑

SYS.1↔SYS.5 · SYS.2↔SYS.4 · SYS.3↔VAL.1 · SWE.1↔SWE.6 · SWE.2↔SWE.5 · SWE.3↔SWE.4

④ 추적성 체인 — STK → SYS → SWR/SAF/HWR → 코드 → 검증

전체 추적성 체인 — V-모델 + ISO 26262



매핑: SAF ↔ TSR / SwSafetyReq · SWR 중 ASIL 항목 = SwSafetyReq · 안전 critical 메시지 (VEH_CTRL/MTR_CMD/ECU_HEARTBEAT) → 검증 시 100% 통과 필수

⑤ 안전 체인 — HARA → SG → FSR → TSR (ISO 26262)

ISO 26262 안전 체인 — ACC 프로젝트 매핑

① HARA
Part 3 Cl.7

H-01
의도치 않은 가속
S2 · E3 · C3 — **ASIL C**

H-02
감속 불가
S3 · E4 · C3 — **ASIL D**

H-03
의도치 않은 급감속
S2 · E3 · C2 — **ASIL B**

근거: SAF.sdoc
시나리오 정의:
acc_iso26262_guide.docx

→ **Confluence: HARA**

② Safety Goals
Part 3 Cl.8

SG-01 (SAF001)
의도치 않은 가속 방지

SG-02 (SAF002)
접근 시 감속
(AEB 미포함)

SG-03 (SAF003)
오버라이드 보장
→ STANDBY 전이

Safe State:
FAULT (모터 0)

FTTI: 50ms

→ **Confluence: SG**

③ FSR (8건)
Part 3 Cl.9

FSR-01 (SAF004)
브레이크 → STANDBY

FSR-02 (SAF005)
센서 고장 → FAULT

FSR-03 (SAF006)
CAN 이상 → FAULT

FSR-04 (SAF007 QM)
목표거리 ↓ 최대감속

FSR-05 (SAF008)
제어출력 범위 제한

FSR-06 (SAF015)
액셀 → STANDBY

FSR-07 (SAF016)
ACC ON 자가진단

FSR-08 (SAF017)
페달 입력 시 활성화 거부

→ **SYS.2/SWR 분배**

④ TSR (6건)
Part 4 Cl.6

TSR-02 (SAF010)
MTR_CMD E2E P01

TSR-03 (SAF011)
워치독 타이머

TSR-04 (SAF012)
프로그램 흐름 모니터

TSR-05 (SAF013)
RAM 무결성 검사

TSR-06 (SAF014)
모터 독립 차단 경로

TSR-07 (SAF018)
하트비트 모니터링

CAN: 0x210 + E2E
CAN: 0x480 (10ms HB)

→ **HW/SW 할당**
HSI 가정 명시

⑤ 구현·검증
Part 6

SWE.3 구현
ACC_ARDUINO/
ACC-RPI/
ACC1 ECU AUTOSAR

SWE.4 단위
MC/DC ASIL B/D
MISRA-C

SWE.5/6 통합·자격
폴트 주입
B2B 테스트

VAL.1
SIL 시나리오
Safety Case

→ **M5/M6**
최종 CR

독립성 I1: 작성자 ≠ 리뷰어 (D = 품질/안전 엔지니어가 A/B/C 산출물 Confirmation Review)

⑥ 프로세스 적용 범위 — ASPICE 4.0 PRM

ASPICE 4.0 — Process Reference Model (PRM)

노란색 = 본 프로젝트 적용 (19 프로세스) · 빗금 = 제외 (테일러링)

System Engineering (SYS)
- SYS.1 Requirements Elicitation · ✅ Full
- SYS.2 Sys Req Analysis ✅ Full
- SYS.3 Sys Architectural Design · ✅ Full
- SYS.4 Sys Integration ⚠ Partial (HIL 대체)
- SYS.5 Sys Verification ⚠ Partial

Software Engineering (SWE)
- SWE.1 SW Req Analysis ✅ Full
- SWE.2 SW Architecture ✅ Full
- SWE.3 Detailed Design + Construction · ✅ Full
- SWE.4 SW Unit Verif. ✅ Full · MC/DC
- SWE.5 SW Integration ✅ Full
- SWE.6 SW Verification ✅ Full

Hardware Engineering (HW)
- HW.1 HW Req Analysis ⚪ Light (HSI)
- HW.2 HW Design ⚪ Light · 결선/핀맵
- HW.3 HW Verification ⚪ Light · VT-01~08
- HW.4 HW Integration ⚪ Light · HW-INT-01~06

Validation (VAL)
- VAL.1 Validation ⚠ Partial · 시나리오

Management (MAN)
- MAN.3 Project Mgmt ✅ Full
- MAN.5 Risk Mgmt ⚪ Light
- MAN.6 Measurement ⚪ Light

Supporting (SUP)
- SUP.1 QA ✅ Full
- SUP.8 CM ✅ Full
- SUP.9 PRM ✅ Full
- SUP.10 CR ✅ Full
- SUP.11 ML 제외

▶ 제외 프로세스 (테일러링)
- MAN.4 Supplier — 공급자 없음 (외주 / 계약 X)
- ACQ.* Acquisition — 획득/구매 활동 없음
- SPL.* Supply — 고객 / 납품 없음
- REU.* Reuse — 재사용 자산 라이브러리 부재
- PIM.* Process Improvement — 학습 프로젝트

▶ Machine Learning Engineering (MLE) — 제외
- MLE.1 ML Req — YOLO 사전학습 모델 사용
- MLE.2 ML Architecture — 자체 학습 X
- MLE.3 ML Training
- MLE.4 ML Model Testing

적용 19 / 제외 다수 — 본 프로젝트 (4주, 5인) 테일러링 결과
SYS 5 + SWE 6 + HW 4 (Light) + MAN 3 + SUP 4 + VAL 1 = **19 적용** | 제외: MAN.4 · ACQ · SPL · REU · PIM · SUP.11 · MLE.1~4 (학습 프로젝트 / 외주 없음 / 사전학습 모델 사용)

Primary Lifecycle (SYS / SWE / HW / VAL) · Organizational (MAN) · Supporting (SUP) · 제외 (테일러링)

📋 목차

00. OVERVIEW

0.1 테일러링 선언서

0.2 프로젝트 개요

01. MAN

1.1 MAN.3 프로젝트 관리

1.2 MAN.5 리스크 관리

1.3 MAN.6 측정

02. SYS

2.1 SYS.1 요구사항 도출

2.2 SYS.2 시스템 요구사항 분석

2.3 SYS.3 시스템 아키텍처 설계

2.4 SYS.4-5 시스템 테스트 (통합+자격)

03. SWE

3.1 SWE.1 SW 요구사항 분석

3.2 SWE.2 SW 아키텍처 설계

3.3 SWE.3 SW 상세설계 및 구현

3.4 SWE.4-6 SW 검증·통합·자격 시험

04. HW

4.1 HW.1 HW 요구사항

4.2 HW.2 HW 설계 (결선도+보드)

4.3 HW.3 HW 검증

4.4 HW.4 HW 통합 시험

05. SUP

5.1 SUP.1 품질 보증

5.2 SUP.8 형상 관리

5.3 SUP.9 문제 해결 관리

5.4 SUP.10 변경 요청 관리

06. VAL

6.1 VAL.1 밸리데이션

07. TRACE

7.1 요구사항 추적 매트릭스 (RTM)

7.2 안전 체인 (ISO 26262)

테일러링 선언서

ASPICE 4.0 + ISO 26262:2018 적용 / 제외 결정

적용 표준 및 적용 범위

적용 표준	Automotive SPICE 4.0 (PAM v4.0) + ISO 26262:2018
대상 시스템	1/10 스케일 RC카 ACC 시스템 (Adaptive Cruise Control)
목표 Capability Level	Core 프로세스 CL3 (Established), 보조 프로세스 CL2 (Managed)
독립성 레벨	I1 — 동일 팀 내 cross-review
최고 ASIL	ASIL D (H-02 감속 불가)

포함 프로세스 (적용 19개)

그룹	프로세스	적용 수준	목표 CL
MAN	MAN.3 프로젝트 관리	Full	CL3
	MAN.5 리스크 관리	Light	CL2
	MAN.6 측정	Light	CL2
SYS	SYS.1 요구사항 도출	Full	CL3
	SYS.2 시스템 요구사항 분석	Full	CL3
	SYS.3 시스템 아키텍처	Full	CL3
	SYS.4 시스템 통합 테스트	Partial	CL2
	SYS.5 시스템 자격 테스트	Partial	CL2
SWE	SWE.1 SW 요구사항 분석	Full	CL3
	SWE.2 SW 아키텍처	Full	CL3
	SWE.3 SW 상세설계·구현	Full	CL3
	SWE.4 SW 단위 검증	Full	CL3
	SWE.5 SW 통합 테스트	Full	CL3
	SWE.6 SW 자격 테스트	Full	CL3
HW	HW.1 요구사항	Light	—
	HW.2 설계	Light	—
	HW.3 검증	Light	—
	HW.4 통합 테스트	Light	—
SUP	SUP.1 품질 보증	Full	CL3
	SUP.8 형상 관리	Full	CL3
	SUP.9 문제 해결	Full	CL3
	SUP.10 변경 관리	Full	CL3
VAL	VAL.1 밸리데이션	Partial	CL2

제외 프로세스 및 사유

프로세스	제외 사유
MAN.4 (공급자 관리)	외주/공급자 없음 — 학습 프로젝트
ACQ.* (획득)	구매·계약 활동 없음
SPL.* (공급)	고객 없음

REU.* (재사용)	재사용 자산 라이브러리 부재
PIM.* (프로세스 개선)	비양산 / 학습 목적
ISO 26262 Part 5 (HW)	범용 MCU 사용, HSI 가정 명시로 대체

테일러링 결정 근거: 본 프로젝트는 양산이 아닌 새싹(SeSAC) 학습 프로젝트로 5인 1개월 일정에 맞춰 핵심 V-모델(SYS / SWE) 프로세스를 Full / CL3 수준으로 적용하고, 보조 프로세스(MAN.5/6, VAL.1)는 Light/Partial 로 운영한다. HW 그룹은 범용 MCU·아두이노 사용으로 ISO 26262 Part 5 의 HW 안전 분석을 가정 명시 방식으로 갈음한다.

0.2 · 00. OVERVIEW

↑ 목차로

ACC 프로젝트 개요

새싹(SeSAC) AUTOSAR 기반 1/10 스케일 RC카 ACC 시스템

프로젝트 정보

프로젝트명	1/10 스케일 RC카 기반 ACC 시스템
목적	AUTOSAR 기반 ACC SW 개발 프로세스 학습 및 포트폴리오
팀 규모	5명
프로젝트 기간	~ 2026.04.30
HW 플랫폼	NXP TRK-MPC5606B · Arduino · Raspberry Pi 5
적용 표준	Automotive SPICE 4.0 + ISO 26262:2018
목표 Capability Level	CL3 Established

주요 기능 요구사항

- 전방 차량 인식 후 거리/속도 자동 제어 SWC 구현
- 시간 기반 안전거리 모델 (Short / Medium / Long 3단계)
- ACC 상태 관리 (OFF → STANDBY → CRUISING / FOLLOWING)
- 운전자 오버라이드 (브레이크 / 액셀) 즉시 반응 (≤50ms)
- 고장 안전 (FAULT 시 모터 출력 0, 자연 감속)
- 통합 시나리오 테스트

하드웨어 구성

구성 요소	사양	역할
메인 ECU	NXP TRK-MPC5606B	ACC 제어 로직 (AUTOSAR)
보조 MCU	Arduino	모터 제어, 센서 인터페이스
호스트	Raspberry Pi 5	HMI GUI, 카메라 처리
거리 센서	LiDAR (MRR-30)	전방 차량 거리 측정
카메라	RPi Camera	전방 차량 인식
통신 버스	CAN	ECU 간 메시지 통신

요구사항 통계

유형	개수	원본
이해관계자 요구사항 (STK)	24	`reqs/STK.sdoc`
시스템 요구사항 (SYS)	37	`reqs/STK/SYS.sdoc`
SW 요구사항 (SWR)	30	`reqs/STK/SYS/SWR.sdoc`

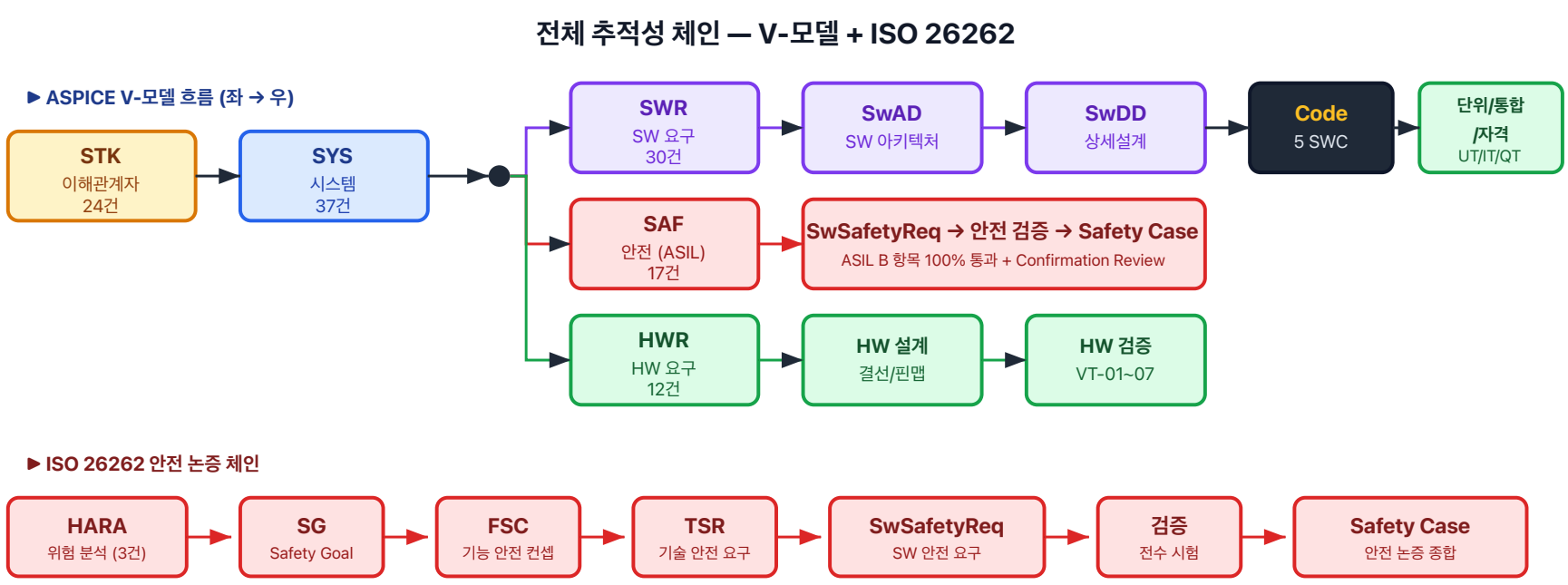
안전 요구사항 (SAF)	17	`reqs/STK/SYS/SAF.sdoc`
HW 요구사항 (HWR)	12	`reqs/STK/SYS/HWR.sdoc`

안전성 (HARA)

Hazard ID	시나리오	ASIL
H-01	의도치 않은 가속	C
H-02	브레이크 미작동	D
H-03	의도치 않은 급감속	B

ASIL D 항목으로 인해 ISO 26262 Part 6 ASIL D 기법 적용: MC/DC 100%, MISRA-C 필수, 폴트 주입 필수, 방어적 프로그래밍 필수.

전체 추적성 체인



매핑: SAF ↔ TSR / SwSafetyReq · SWR 중 ASIL 항목 = SwSafetyReq · 안전 critical 메시지 (VEH_CTRL/MTR_CMD/ECU_HEARTBEAT) → 검증 시 100% 통과 필수

MAN.3 프로젝트 관리

Automotive SPICE 4.0 — Project Management · Full / CL3

MAN.3 Project Management

항목	포함 내용
프로젝트 범위	ACC 시스템의 개발 범위, 목표 차량/플랫폼, 기능 경계 정의
WBS / 일정	4주 일정에 맞춘 Work Breakdown Structure, 마일스톤 (SRR, PDR, CDR, 검증 완료)
역할 / 책임	4명의 R&R 매트릭스 (시스템 담당, SW담당, HW 담당, 안전/품질 담당)
리소스 계획	도구(DOORS/Polarion, MATLAB, 컴파일러, HIL 장비), 환경, 예산
위험 관리	기술적/일정 리스크 식별, 영향도 평가, 대응 전략
의사소통 계획	회의주기(일일 스탠드업, 주간 리뷰), 보고 체계
ISO 연계: Safety Plan	안전 활동 일정, 안전 관리자 지정 확인 리뷰(Confirmation Review) 계획

8. 2026-04-30 현재 상태 업데이트

8.1 프로젝트 범위 (확정)

- 시스템: 1/10 스케일 RC카 기반 Adaptive Cruise Control (STK001)
- 타깃 HW: NXP MPC5606B (메인 ECU) + Raspberry Pi 5 (센서/HMI) + Arduino Uno ×2 (CAN 게이트웨이 + 4륜 모터)
- 기능 범위: 전방차 인식(카메라+LiDAR+YOLO), 차간거리 제어, 5상태 머신(OFF/STANDBY/CRUISING/FOLLOWING/FAULT), 운전자 오버라이드(브레이크/액셀)

8.2 R&R 매트릭스 (Jira 기반 실제 담당)

영역	산출물/모듈	담당
요구사항 (STK/SYS)	ACC-docs/reqs/STK.sdoc, ASPICE/SYS1.sdoc, ASPICE/SYS2.sdoc	요구사항팀
HMI (PyQt5)	ACC-RPI/acc_hmi/	HMI 담당
센서 융합	ACC-RPI/acc_fusion/ (Camera+LiDAR+YOLO)	Fusion 담당
CAN 모듈 (Python)	ACC-RPI/acc_can/ (cantools + DBC-driven)	CAN 담당
Arduino CAN 게이트웨이	ACC_ARDUINO/acc_can_node/	Arduino 담당
Arduino 모터 제어	ACC_ARDUINO/acc_motor_node/	Arduino 담당
ECU 진단/제어 모델	ACC_Diagnostics.slx, ACC_Types.sldd	ECU 담당
QA / Confirmation Review	(SUP.1 참조)	김유림(PM)

8.3 WBS 진행 현황 (Jira ACC 프로젝트 기준)

워크 패키지	Jira	상태	산출물 위치
REQUIREMENTS Epic	ACC-16	해야 할 일(Epic)	—
└ SYS1	ACC-17	✅ 완료	ASPICE/SYS1.sdoc (14 stakeholders)
└ SYS2	ACC-18	🟡 진행 중	ASPICE/SYS2.sdoc (20 reqs)
└ SWE1	ACC-19	⏳ 해야 할 일	(미작성)
요구사항 합의 업데이트	ACC-21	✅ 완료	—
doorstop → strictdoc 마이그레이션	ACC-22	✅ 완료	—
github pages 배포	ACC-23	✅ 완료	—
간단 HMI 구현	ACC-24	✅ 완료	—
요구사항 업데이트 확인	ACC-25	⏳ 해야 할 일	—
문서작업 Epic	ACC-20	해야 할 일(Epic)	—

└ SUP1/8/9/10 문서작성	ACC-26/28/29/30	✅ 완료	Confluence
HMI 모듈 코드작성	ACC-31	🟡 진행 중	ACC-RPI/acc_hmi/
Fusion 모듈 코드작성	ACC-32	✅ 완료	ACC-RPI/acc_fusion/
Arduino 모터 코드작성	ACC-33	🟡 진행 중	ACC_ARDUINO/acc_motor_node/
CAN 모듈 코드작성	ACC-34	🟡 진행 중	ACC-RPI/acc_can/, ACC_ARDUINO/acc_can_node/
MAN5/MAN6	ACC-6/7	✅ 완료	(이 페이지 외)
MAN3	ACC-5	✅ 완료 (v1.1, 2026-04-30)	본 문서

8.4 리소스 / 도구 (실제 사용 도구)

분류	실제 사용 도구	비고
요구사항 관리	StrictDoc (.sdoc) — DOORS/Polarion 미사용	doorstop → strictdoc 마이그레이션 완료 (ACC-22)
MBD	MATLAB / Simulink (ACC_Diagnostics.slx)	ECU 진단 모델
모델 데이터	ACC_Types.sldd	Simulink Data Dictionary
CAN DB	DBC 파일 (ACC-CANDB/Project.dbc + acc_db.dbc)	cantools 로 RPi 측 로드
AUTOSAR	mobilgene (Project-original.dbc 베이스)	NM/TP/IF 풀셋 포함
코드 (RPi)	Python 3.13, uv, PyQt5, cantools, Hailo NPU SDK	
코드 (Arduino)	Arduino IDE, DFRobot_MCP2515, Wire	Uno × 2
이슈/형상	Jira (gaepo) + GitHub (ACC-RPI, ACC_ARDUINO, ACC-CANDB, ACC-docs)	
문서	Confluence (gaepo) — MAN/SUP	

8.5 위험 관리 — 신규 식별 (MAN.5 와 동기화)

1. ~~CAN ID 재배정 후 일부 리포지토리에 옛 ID 잔존 → 통합 시험 시 ID 미스매치 위험~~ — **해소 (2026-04-30)**: `Project.dbc` 정본 9 메시지 ID/주기 검증 완료, 발표 자료 슬라이드 27 (CAN 메시지 흐름) 도 정본 기준으로 재작성.
2. STK019 의 9종 버튼 정의 vs DBC ACC_CTRL 의 4 BTN 시그널 → 매핑 룰 미합의 위험
3. 단일 엔코더 → 4채널 MTR_SPD_FB 브로드캐스트 → 추후 4엔코더 확장 결정 필요
4. SWE.1 미작성으로 코드 ↔ SYS.2 트레이스 단절 위험 → **완화 (2026-04-30)**: SWE.1 코드 ↔ SWR 매핑 표에 5 SWC Runnable 명(`RE_FsmStep` / `RE_PidStep` / `RE_MotorCtrlStep` / `RE_FaultDetect` / `RE_HmiStep`) 추가 반영.
5. **(신규)** MotorControl PI 게인 `KP/KI` 및 DistanceControl `DIST_KP/KI/KD` 가 `TODO_TUNE` 상태 — HIL/벤치 튜닝 미진행 시 정상 상태 오차 측정 갭. 시연일까지 잠정값 유지.
6. **(신규)** MTR_CMD E2E P01 검증 — Arduino 수신측 활성화 OK, ECU 송신측은 placeholder. SAF010 부분 적용 (DBC CM 에 TODO 명시).

8.6 의사소통 계획 보강

- 일일 스탠드업: SUP.9 §5 "일일 추적" 과 동기화 (Jira 보드 OPEN/IN PROGRESS 검토)
- 주간 리뷰: 본 MAN.3 + SUP.10 (CR 이력 요약) + MAN.6 (메트릭) 통합 보고

MAN.5 리스크 관리

Automotive SPICE 4.0 — Risk Management · Light / CL2

항목	포함 내용
리스크 식별	기술 리스크, 일정 리스크, 인력 리스크, 품질/프로세스 리스크, 외부환경 리스크 목록
리스크 분석	발생 확률 × 영향도 매트릭스, 우선순위 결정
완화 전략	각 리스크에 대한 회피/전가/완화/수용 전략
모니터링	리스크 상태 추적 주기, 트리거 조건, 에스컬레이션 기준
ISO 연계	안전 관련 리스크는 Safety Plan의 안전 활동과 연결

6. 2026-04-30 신규 식별 리스크 (코드/DBC 분석 결과)

워크스페이스의 요구사항(STK.sdoc, SYS2.sdoc), CAN DB(Project.dbc), 코드(ACC-RPI, ACC_ARDUINO)를 자동 분석한 결과 식별된 리스크. 발생 확률(L/M/H) × 영향도(L/M/H) 매트릭스에서 **High** 4건, **Medium** 2건이 신규 등재됨.

6.1 신규 리스크 등록부

ID	리스크	분류	확률	영향	등급	트리거	완화 전략	담당	연계 CR
RISK-2026-001	**CAN ID 정합성 단절** — Project.dbc 머지로 MTR_SPD_FB 0x300→0x310, MTR_HEARTBEAT 0x310→0x320, ECU_HEARTBEAT 0x410→0x480 재배정. 4개 파일에 옛 ID 잔존	기술 (인터페이스)	M	H	**High**	통합 시험에서 ECU↔Arduino 통신 실패	_dbc.py / CLAUDE.md / Arduino config.h / Arduino README 갱신	CAN 담당 (ACC-34)	CR-DOC + CR-SW
RISK-2026-002	**Arduino MTR_CMD 페이로드 포맷 미스매치** — 옛 6바이트(checksum) vs 현재 DBC E2E P01 (4×PWM + MTR_RC + MTR_CRC)	기술 (안전)	M	H	**High**	E2E 검증 실패 → 모터 safe-state, 무검증 시 SAF010 ASIL-B 위반	can_handler.cpp 코드리뷰 → 수정 + 회귀 테스트	Arduino 담당 (ACC-33/34)	CR-SAF
RISK-2026-003	**STK019 ↔ DBC 버튼 매핑 룰 미합의** — STK019 9종 vs DBC ACC_CTRL 4 BTN	요구사항/설계	H	M	**High**	HMI 인코딩 모호 → 모듈 간 동작 불일치	SYS.2 IF에 STK019 ↔ ACC_CTRL 매핑 명시	요구사항팀 + HMI/CAN	CR-REQ
RISK-2026-004	**단일 엔코더 → 4채널 broadcast 한계** — 엔코더 1개 vs DBC 4채널	기술/HW	M	M	**Medium**	4엔코더 확장 시 변경 광범위	STK/SYS.2에 1-엔코더 한계 + backlog 이관	HW + 요구사항	CR-REQ
RISK-2026-005	**SWE.1 미작성으로 트레이스 단절** — ASPICE/SWE1.sdoc 부재	프로세스 (ASPICE)	H	M	**High**	ASPICE Assessment NC, 영향 분석 불가	ACC-19 즉시 착수 + SWR-NNN 발급	요구사항	—
RISK-2026-006	**L/R 2채널 변환 미합의** — 모터 L/R 2 PWM vs DBC 4 PWM	설계	M	M	**Medium**	4륜 독립 제어 시 adapter 재설계	SYS.2 IF-001로 adapter 동작 명시	CAN + 요구사항	CR-DES

6.2 매트릭스 (확률 × 영향)

	영향 L	영향 M	영향 H
확률 H	—	RISK-005	RISK-003
확률 M	—	RISK-004, 006	RISK-001, 002
확률 L	—	—	—

6.3 모니터링 / 에스컬레이션

- **주간**: 본 등록부의 상태 (Open/Mitigating/Closed) 검토 — 주간 리뷰에서 갱신
- **트리거**: RISK-002 / RISK-005 의 등급은 안전 관련(ASIL-B) 이므로 **에스컬레이션 24h 이내** (SUP.9 §5 P1 이슈와 동일 처리)
- **종결 조건**: 연계 CR 이 SUP.10 절차에 따라 Closed 되고 회귀 테스트 통과 시 본 등록부 Closed 처리

6.4 ISO 26262 연계

- RISK-002 (E2E P01 미스매치) 는 SAF010 (TSR-02) 와 직접 연관 — Confirmation Review 대상
- RISK-005 (SWE.1 미작성) 은 Part 6 §7 (Software safety requirements) 추적성 갭

1.3 · 01. MAN

↑ 목차로

MAN.6 측정

Automotive SPICE 4.0 — Measurement · Light / CL2

항목	포함 내용
측정 목표	프로젝트 품질/진도 파악을 위한 측정 목표 정의

측정 항목(Metrics)	요구사항 커버리지율, 결함 밀도, 테스트 통과율, 코드 커버리지(MC/DC), 일정 준수율
수집 방법	각 메트릭의 수집 시점, 도구, 담당자
분석/보고	주간 대시보드, 이상치 대응 절차

5. 2026-04-30 측정 스냅샷

5.1 요구사항 커버리지

산출물	항목 수	출처
STK (이해관계자 요구사항)	**24** REQUIREMENT (STK001~STK024)	ACC-docs/reqs/STK.sdoc
SYS.1 stakeholders	**14** (SYS1-STK-001~014)	ACC-docs/ASPICE/SYS1.sdoc
SYS.2 요구사항	**20** (FR ×6, PR ×1, SR ×2, IF ×2, NF ×1, AT ×3, HARA ×1, SG ×1, FSC ×1, QA ×2)	ACC-docs/ASPICE/SYS2.sdoc
SWE.1 요구사항	**0** (미작성)	—

STK → SYS1 추적성: STK.sdoc 의 모든 REQUIREMENT 가 SYS1-STK-xxx 를 Parent 로 보유 → 100%
SYS2 ↔ STK / SWE1 추적성: 양방향 트레이스 미보강 → **갭 존재**

5.2 일정 준수율 (Jira 기반)

Epic / 그룹	총 작업	완료	진행 중	해야 할 일	완료율
문서작업 (ACC-20 + 자식)	5	5	0	0	**100%**
REQUIREMENTS (ACC-16 + 자식)	4	1 (SYS1)	1 (SYS2)	2 (SWE1, Epic)	25%
코드 작업 (ACC-31~34)	4	1 (Fusion)	3 (HMI, Arduino, CAN)	0	25%
MAN/SUP Epics (ACC-5~11)	7	4 (MAN5/6, SUP1, 외)	0	3	57%
기타 단발 작업 (ACC-21~25)	5	4	0	1 (ACC-25)	80%
전체 (Project ACC, 25개 이슈)	25	14	4	7	**56%**

5.3 형상 항목 카운트

형상 항목	개수 / 위치
StrictDoc 문서	3 (STK.sdoc, SYS1.sdoc, SYS2.sdoc)
Confluence 페이지 (MAN/SUP)	7 (MAN.3/5/6, SUP.1/8/9/10)
CAN DB 파일	3 (acc_db.dbc, Project.dbc, Project-original.dbc)
CAN 메시지 (ACC 9 + Project 38)	acc_db: 9, Project: 38 (ACC 9 + 베이스 29)
Python 모듈 (ACC-RPI)	3 (acc_can, acc_hmi, acc_fusion) + interfaces/common
Arduino 노드	2 (acc_can_node, acc_motor_node)
Simulink 모델	1 (ACC_Diagnostics.slx)

5.4 결함 밀도 / 정합성 NC

NC ID	영역	등급	비고
NC-001	CAN ID 주석/문서 옛 ID 잔존 (4개 파일)	Major	RISK-2026-001
NC-002	Arduino MTR_CMD 페이로드 포맷 미스매치	Critical	RISK-2026-002 (ASIL-B)
NC-003	STK019 ↔ DBC 버튼 매핑 룰 부재	Major	RISK-2026-003
NC-004	SWE.1 산출물 미작성 (트레이스 단절)	Major	RISK-2026-005
NC-005	1-엔코더로 4채널 broadcast 비명문화	Minor	RISK-2026-004
NC-006	L/R 어댑터 변환 미합의	Minor	RISK-2026-006

→ **결함 밀도**: 6 NC / 56 형상 항목 ≈ **10.7%** (목표: < 5%, **이상치 — MAN.5 / SUP.10 트리거**)

5.5 다음 측정 주기 액션

- 주간 리뷰에서 5.4 의 NC 처리 진행률을 갱신 (목표: 1주 내 NC-001/002 Critical/Major Closed)
- SWE.1 작성 진척도(0% → 목표 30%) 신규 메트릭 추가

- ACC-RPI / ACC_ARDUINO 의 CI 도입 여부 점검

SYS.1 요구사항 도출

Stakeholder Requirements — STK + 14 stakeholders

핵심

- **이해관계자**: 운전자 / 개발팀 4인 / SESAC 멘토 / 안전·HW·도구 공급자 / UNECE / ISO 26262 인증 / Euro NCAP
- **도출 방법**: ISO 15622·22179 표준 분석, UNECE WP.29, 경쟁사 벤치마킹, 팀 회의 + 멘토 리뷰, 사용 시나리오 (CRUISING / FOLLOWING / 오버라이드 / FAULT)

STK 요구사항 (전체 24건)

UID	제목
STK001	시스템 목적
STK002	전방 차량 인식
STK003	차간거리 측정
STK004	속도 제어
STK005	순항 속도 설정
STK006	안전거리 단계 설정
STK007	운전자 오버라이드 (브레이크)
STK008	운전자 오버라이드 (가속 → STANDBY)
STK009	ACC 상태 관리
STK010	고장 안전 (FAULT 동작)
STK011	운영 환경
STK012	통신 인터페이스
STK013	제약 조건 - MCU
STK014	제약 조건 - 개발 표준
STK015	제약 조건 - 개발 일정
STK023	AEB 미포함
STK016	자차 속도 측정
STK017	전방 차량 속도 추정
STK018	ACC SET/RES 재활성화 기능
STK019	운전자 입력 버튼 정의
STK020	HMI 가상 GUI 인터페이스
STK021	버튼 가용성 규칙
STK022	ACC ON 동작 흐름
STK024	최소 설정 속도

적용

Full / CL3 · ✅ Jira ACC-17 완료 · GitHub Pages 자동 배포 (ACC-23)

SYS.2 시스템 요구사항 분석

System Requirements — SYS + SYS2 분석

핵심

- **섹션**: 상태 전이 / 운전자 오버라이드 / 안전거리 모델 / 인식·측정 / 제어·성능 / CAN 통신 / 운전자 인터페이스 / 비기능
- **SYS2 분석 카테고리**: FR×6 · PR×1 · SR×2 · IF×2 · NF×1 · AT×3 · HARA×1 · SG×1 · FSC×1 · QA×2
- **추적성**: STK ↔ SYS Parent 100% (RELATIONS 블록)

SYS2 분석 (20건)

UID	제목
SYS2-FR-001	상태 전이 완전성 분석
SYS2-FR-002	운전자 오버라이드 요구사항 분석
SYS2-FR-003	안전거리 모델 분석
SYS2-FR-004	인식/측정 요구사항 분석
SYS2-FR-005	운전자 인터페이스 요구사항 분석
SYS2-FR-006	기타 기능 요구사항 분석
SYS2-PR-001	제어 성능 요구사항 분석
SYS2-SR-001	FSR 분석 요약
SYS2-SR-002	TSR 분석 요약
SYS2-IF-001	CAN 통신 인터페이스 매트릭스
SYS2-IF-002	센서 데이터 포맷 분석
SYS2-NF-001	비기능 요구사항 분석
SYS2-AT-001	요구사항 속성 체계
SYS2-AT-002	요구사항 카테고리별 분포
SYS2-AT-003	검증 방법 매트릭스
SYS2-HARA-001	HARA 결과 요약
SYS2-SG-001	Safety Goals 요약 및 분석
SYS2-FSC-001	Functional Safety Concept 분석
SYS2-QA-001	완전성 검토 결과
SYS2-QA-002	일관성 검토 결과

SYS 시스템 요구사항 (전체 37건)

UID	제목
SYS001	ACC 상태 전이 - OFF
SYS002	ACC 상태 전이 - STANDBY
SYS003	ACC 상태 전이 - CRUISING
SYS004	ACC 상태 전이 - FOLLOWING
SYS005	ACC 상태 전이 - FAULT
SYS006	브레이크 오버라이드 (→ STANDBY)
SYS007	가속 오버라이드 (→ STANDBY)

SYS008	안전거리 모델 - 단계 1
SYS009	안전거리 모델 - 단계 2
SYS010	안전거리 모델 - 단계 3
SYS035	안전거리 모델 - 최소 안전거리
SYS011	전방 차량 인식 처리
SYS012	자차 속도 측정
SYS013	전방 차량 상대속도 산출
SYS014	PID 속도/거리 제어 (cascaded)
SYS020	제어 응답 시간
SYS021	센서 데이터 갱신 주기
SYS022	차간거리 접근 시 제어 (AEB 미포함)
SYS015	CAN 통신 - 센서 → 메인 ECU
SYS016	CAN 통신 - 메인 ECU → 모터 제어기
SYS017	CAN 통신 - 모터 제어기 → 메인 ECU
SYS025	CAN 통신 고장 감지 (→ FAULT)
SYS030	CAN 통신 - GUI → 메인 ECU (운전자 입력)
SYS031	CAN 통신 - 메인 ECU → GUI (HMI 표시 데이터)
SYS018	운전자 입력 인터페이스 (가상 GUI)
SYS019	HMI 출력 인터페이스 (GUI 클러스터)
SYS029	버튼 가용성 관리
SYS036	MotorControl SWC 도입 (inner-loop 속도 PI)
SYS037	OFF/STANDBY 상태 수동 액셀 제어
SYS023	AUTOSAR 준수
SYS024	MBD 기반 제어 설계
SYS026	직진 전용 차량
SYS027	SET/RES에 의한 STANDBY 복귀
SYS028	일시정지 버튼 동작
SYS032	최소 설정 속도 제한
SYS034	ACC 활성화 시 기본 안전거리 단계
SYS033	페달 입력 중 ACC 활성화 거부

적용

Full / CL3 · 🟡 Jira ACC-18 진행 중

2.3 · 02. SYS

↑ 목차로

SYS.3 시스템 아키텍처 설계

3 노드 분산 + 5 SWC + CAN ICD

핵심 — 3 노드 분산 아키텍처

```
SENSOR (RPi5)  ── CAN 500k ──  ECU (MPC5606B AUTOSAR)  ── CAN ──  MTR (Arduino #1)
+ Camera + LiDAR + Hailo NPU      5 SWC (ASW)                  ── I2C ──  Motor (Arduino #2)
+ HMI (PyQt5)                                                              + Encoder
```

5 SWC + Runnable (ECU 측 — ACC-autosar/mbd/)

SWC	Runnable	주기	책임	ASIL
`AccStateMachine`	`RE_FsmStep`	10ms	5상태 FSM (OFF/STANDBY/CRUISING/FOLLOWING/FAULT) — Stateflow	ASIL-B mixed
`DistanceControl`	`RE_PidStep`	20ms	outer-loop 거리 PID (SWR009/011)	ASIL-B / QM
`MotorControl`	`RE_MotorCtrlStep`	10ms	inner-loop 속도 PI + 4 PWM fan-out + STANDBY 수동 PWM	ASIL-B
`Diagnostics`	`RE_FaultDetect`	10ms	HB 모니터 + Fault 우선순위 + ECU HB 송신 (SAF018)	ASIL-B / QM
`HmiManager`	`RE_HmiStep`	50ms	4 BTN edge-detect + pass-through	QM

Pattern B 적용 — SWC 포트가 DBC ISignal 과 직접 정합되어 mobilgene 이 SystemMapping 자동 생성. CAN 게이트웨이 SWC 없음. 5 Runnable 모두 단일 `OsTask_ASW_FG1_10ms` 에 매핑되며, Timing Event 주기(10/20/50ms)로 RTE sub-rate 동작.

CAN ICD (정본: `ACC-CANDB/Project.dbc`)

ID	메시지	Tx → Rx	DLC	주기	주요 시그널	안전
0x110	SENSOR_FUSION	SENSOR → ECU	3	20 ms	VEH_DET, VEH_DIST(mm)	QM
0x111	SENSOR_HEARTBEAT	SENSOR → ECU	2	20 ms	HB_SENSOR (rolling)	QM
0x120	VEH_CTRL	SENSOR → ECU	2	20 ms	BTN_BRAKE, SET_ACCEL_PWM	**ASIL-B**
0x210	MTR_CMD	ECU → MTR	6	10 ms	SET_PWM_LF/RF/LR/RR + RC + CRC	**ASIL-B + E2E P01**
0x310	MTR_SPD_FB	MTR → ECU, SENSOR	8	10 ms	GET_SPD_AVG (sint16, scale 0.02 cm/s)	QM
0x320	MTR_HEARTBEAT	MTR → ECU	2	10 ms	HB_MTR (rolling)	QM
0x480	ECU_HEARTBEAT	ECU → SENSOR, MTR	2	10 ms	HB_ECU (rolling, broadcast)	**ASIL-B** (SAF018)
0x510	ACC_CTRL	SENSOR → ECU	4	50 ms	BTN_ACC_OFF/SET/RES/PAUSE, SET_ACC_LVL, SET_SPEED	QM
0x520	ACC_STATUS	ECU → SENSOR	4	50 ms	GET_ACC_STATE, ERR_ECU, CUR_SPD, DIST_LVL	QM

총 9 메시지 — Project.dbc `GenMsgCycleTime` 속성 기준 검증 완료 (2026-04-30). CAN 2.0B · 500 kbps. 모든 메시지가 적용 범위 내, NM/TP 등 비-ACC 메시지는 전송 ACL 에서 제외.

FSM 5상태 (STK009 / SYS001~005)

```
OFF ─[ACC ON 토글 + 자가진단]─► STANDBY ─[SET / RES]─► CRUISING ⇄ FOLLOWING
                                   ▲                      │
                                   └──[브레이크/액셀]──────┘
* → FAULT (센서/통신 고장 SYS025) → 운전자 OFF 토글로만 복구 (능동 감속 없음)
```

적용

Full / CL3 · DBC + ARXML(mobilgene) 으로 형상 통제. RootComposition 에 `AccComposition` 등록 + EcuExtract 가 5 SWC 평면 노출.

SYS.4-5 시스템 테스트 (통합 + 자격)

System Integration + Qualification — 보텀업 4단계 + M5/M6 시연 4 시나리오

핵심

- **SYS.4 통합 시험** + **SYS.5 자격 시험** 통합 운영 (Partial/CL2). HIL 부재로 벤치+실차체로 갈음.
- 출처: `ACC_ARDUINO/README.md` § 테스트 절차, `ACC-autosar/RUNBOOK.md` , M5/M6 시연 시나리오

통합 단계 (보텀업 4단계)

단계	대상	방법
Stage 1	Arduino #2 (모터 단독)	임시 코드로 PWM 80 인가 → 4륜 정/역 회전 확인 + 엔코더 PPR 측정 (DEBUG_ENCODER=1)
Stage 2	Arduino #1 (CAN 단독)	PCAN-View → `0x210 MTR_CMD` + `0x480 ECU_HEARTBEAT` 송신, 시리얼 로그로 수신 확인
Stage 3	#1 ↔ #2 (I2C 통합)	A4-A4/A5-A5/GND 연결 → 모터 응답 + `0x310 MTR_SPD_FB` + `0x320 MTR_HEARTBEAT` 수신
Stage 4	ECU + RPi 풀 통합	4 노드 풀 시스템 (SENSOR_FUSION + ACC_CTRL → ACC_STATUS + MTR_CMD)

자격 시연 시나리오 (M5/M6)

시나리오	STK / SYS	합격 기준
QT-01 CRUISING 순항 — 전방 차 무	STK001/004/005, SYS003/014	설정 속도 ±2cm/s 유지 ≥ 5초
QT-02 FOLLOWING 추종 — 전방 차 + 거리 1/2/3	STK002/003/006, SYS004/008~011	목표 거리 ±10cm, 단계 변경 시 1초 내 재안정화
QT-03 브레이크 오버라이드	STK007, SYS006, SAF003/004	입력 후 50ms 내 STANDBY + PWM=0
QT-04 FAULT 진입 — SENSOR HB 60ms 끊김	STK009, SYS005/025, SAF018	FAULT + HMI 경고 + 운전자 수동 OFF로만 복구

적용

Partial / CL2 · HIL 없음. 환경: PCAN-USB 로깅 + RPi 자체 로그 + M6 시연 영상.

> SYS.4 와 SYS.5 는 본 프로젝트에서 시연일에 결합 수행하므로 단일 페이지로 통합 운영.

SWE.1 SW 요구사항 분석

Software Requirements — SWR + 코드 매핑

핵심

- **구조**: SW 컴포넌트(SWC) 단위로 그룹화 — ECU 5 SWC + RPi 3 모듈 + Arduino 2 노드
- **추적성**: SWR ↔ SYS Parent 100%

코드 모듈 ↔ SWR 매핑

SWC / 모듈	Runnable	저장소	관련 SWR
AccStateMachine (Stateflow FSM)	`RE_FsmStep` · 10 ms	`ACC-autosar/mbd/AccStateMachine/`	SWR001~003, 026~031
DistanceControl (outer PID)	`RE_PidStep` · 20 ms	`ACC-autosar/mbd/DistanceControl/`	SWR008~011
MotorControl (inner PI + 4 PWM)	`RE_MotorCtrlStep` · 10 ms	`ACC-autosar/mbd/MotorControl/`	SWR032~034
Diagnostics (HB + FAULT)	`RE_FaultDetect` · 10 ms	`ACC-autosar/mbd/Diagnostics/`	SWR015~016
HmiManager (BTN edge)	`RE_HmiStep` · 50 ms	`ACC-autosar/mbd/HmiManager/`	SWR020, 028
RPi `acc_can`	—	`ACC-RPI/acc_can/` (cantools)	SWR012~015, 030
RPi `acc_fusion`	—	`ACC-RPI/acc_fusion/` (Camera+LiDAR+YOLO)	SWR004, 007
RPi `acc_hmi`	—	`ACC-RPI/acc_hmi/` (PyQt5 ~780줄)	SWR018~020
Arduino `acc_can_node`	—	`ACC_ARDUINO/acc_can_node/`	SWR012~015 (Arduino측)
Arduino `acc_motor_node`	—	`ACC_ARDUINO/acc_motor_node/`	SWR032~034 (Arduino측)

5 Runnable 모두 단일 OS Task `OsTask_ASW_FG1_10ms` 에 매핑되며, Runnable 별 Timing Event 주기(10/20/50 ms)로 RTE 가 sub-rate trigger.

SWR 전체 (31건)

UID	제목
`SWR001`	ACC 상태 머신 구현
`SWR002`	브레이크 오버라이드 처리 (→ STANDBY)
`SWR003`	가속 오버라이드 처리 (→ STANDBY)
`SWR026`	SET/RES 버튼 처리
`SWR027`	일시정지 버튼 처리
`SWR028`	버튼 가용성 상태 관리
`SWR029`	ACC ON 자가진단
`SWR031`	최소 설정 속도 클램핑
`SWR004`	LiDAR 거리 데이터 수신
`SWR007`	자차 속도 수신 및 처리
`SWR008`	안전거리 목표값 계산 (AEB 미포함)
`SWR009`	DistanceControl outer-loop 제어 (거리 PID)
`SWR010`	CRUISE 모드 목표속도 직접 전달
`SWR011`	FOLLOWING 모드 거리 제어 (outer-loop)
`SWR012`	CAN TX - 모터 제어 명령 송신
`SWR013`	CAN RX - 센서 데이터 수신

SWR014	CAN RX - 모터 피드백 수신
SWR015	CAN 메시지 타임아웃 감시
SWR016	고장 감지 및 FAULT 전이 (능동 감속 없음)
SWR018	HMI GUI 표시 (클러스터)
SWR019	FAULT 경고 표시
SWR020	운전자 입력 처리 (GUI → CAN)
SWR021	태스크 주기 - 10ms
SWR022	태스크 주기 - 20ms
SWR023	태스크 주기 - 50ms / 100ms
SWR024	MISRA-C:2012 준수
SWR025	AUTOSAR RTE 포트 정의
SWR032	MotorControl SWC 구현 (상태별 제어 분기)
SWR033	OFF/STANDBY 수동 액셀 PWM 제어
SWR034	MotorControl inner-loop 속도 PI 제어
SWR030	GUI-CAN 인터페이스 (RPi측)

적용

Full / CL3

3.2 · 03. SWE

↑ 목차로

SWE.2 SW 아키텍처 설계

Pattern B (SWC 직접 bus-binding) + 5 SWC + RPi interfaces

핵심 — Pattern B (SWC 직접 bus-binding)

CC (CanCommunication) 게이트웨이 SWC 폐기 → 응용 SWC 가 DBC ISignal 과 직접 정합. mobilgene 이 ARXML import 시 SystemMapping 자동 생성 (수동 입력 0건).

- **인터페이스**: 15 SR + 2 CS + 0 MS
- **OS Task**: `OsTask_ASW_FG1_10ms` 단일 — 5 Runnable 모두 같은 task, Timing Event 주기를 10/20/50ms 차등으로 RTE sub-rate trigger
- **Composition**: `AccComposition` 1개, intra-ECU connector 7개

5 SWC + 인터페이스 (ACC-autosar/mbd/Composition/)

SWC	Provided	Required
`AccStateMachine`	IF_AccMode (SR), IF_AccStatus, IF_AccSetpoint, PP_AccStatusBus (→ 0x520)	RP_SensorBus, RP_SpeedBus, RP_VehCtrlBus, IF_DriverReq, IF_DiagStatus
`DistanceControl`	IF_CtrlOutput (DC → MC)	IF_AccMode, IF_AccSetpoint, RP_SensorBus, RP_SpeedBus
`MotorControl`	PP_MtrCmdBus (→ 0x210 + E2E P01)	IF_CtrlOutput, IF_AccStatus, RP_VehCtrlBus (SET_ACCEL_PWM), RP_SpeedBus
`Diagnostics`	IF_DiagStatus (7 DE), PP_EcuHbBus (→ 0x480)	IF_AccMode, RP_SensorHbBus, RP_MtrHbBus
`HmiManager`	IF_DriverReq	RP_AccCtrlBus, RP_VehCtrlBus (BTN_BRAKE, SET_ACCEL_PWM)

RPi 측 모듈 인터페이스 (interfaces/)

- `FusionData` (acc_fusion → acc_can): VEH_DET, VEH_DIST
- `VehicleInfo` (acc_can RX → HMI): current_speed
- `AccInfo` (acc_can RX → HMI): state, set_spd, set_lvl
- `ButtonInput` , `PedalInput` , `AccSetting` , `EcuStatus` (HMI → acc_can TX)

적용

Full / CL3 · ARXML 정형 모델 + Pattern B 자동 매핑

3.3 · 03. SWE

↑ 목차로

SWE.3 SW 상세설계 및 구현

5 SWC 구현 + RPi/Arduino 모듈 + Cascaded 2-loop + E2E P01

핵심 — 모듈별 구현 상태

모듈	Runnable	상태	핵심 파일
AccStateMachine (FSM)	`RE_FsmStep` · 10 ms	**v1.0 안정화**	`mbd/AccStateMachine/implement_fsm_AccStateMachine.m` — 5 leaf + 2 super, outer transition (any→FAULT, 오버라이드→STANDBY)
DistanceControl (outer PID)	`RE_PidStep` · 20 ms	**v1.0**	`mbd/DistanceControl/build_logic_DistanceControl.m`
MotorControl (inner PI)	`RE_MotorCtrlStep` · 10 ms	**v0.7+**	`mbd/MotorControl/build_logic_MotorControl.m` — STANDBY 수동 PWM pass-through, SAF008 클램핑
Diagnostics (HB + FAULT)	`RE_FaultDetect` · 10 ms	**v0.8+**	`mbd/Diagnostics/build_logic_Diagnostics.m` — HB freeze + Fault 우선순위 + ECU HB 송신
HmiManager	`RE_HmiStep` · 50 ms	**v0.8+**	`mbd/HmiManager/create_swc_HmiManager.m` — 4 BTN edge-detect
RPi `acc_fusion`	—	**완료 (ACC-32)**	Camera + LiDAR + YOLO orchestrator
RPi `acc_can`	—	**DBC-driven 리팩터 완료**	`_codec.py` — encode 4 / decode 3 함수
RPi `acc_hmi`	—	**진행 중 (ACC-31)**	`hmi_gui.py` ~780줄 PyQt5
Arduino `acc_can_node`	—	**새 E2E P01 포맷 적용 (ACC-34)**	`can_handler.cpp` — DFRobot_MCP2515 폴링
Arduino `acc_motor_node`	—	**진행 중 (ACC-33)**	`motor_driver.cpp` + `encoder.cpp`

핵심 알고리즘

FSM 전이 (AccStateMachine)

```
OFF —[BTN_ACC_OFF 토글 + 자가진단 OK]—► STANDBY (STK022)
STANDBY —[BTN_ACC_SET]—► CRUISING (현재 속도, STK018)
STANDBY —[BTN_ACC_RES]—► CRUISING/FOLLOWING (이전 속도, STK019)
CRUISING/FOLLOWING —[BTN_BRAKE | SET_ACCEL_PWM > 0]—► STANDBY (50ms, SAF003/004)
* —[HB freeze | err flag]—► FAULT (능동 감속 없음, STK009)
```

제어 (Cascaded 2-loop)

```
DistanceControl (outer, 20ms): 거리 PID — VEH_DIST + d_target → TargetSpeed
MotorControl (inner, 10ms):    속도 PI — TargetSpeed + GET_SPD_AVG → MTR_CMD 4 PWM
                                STANDBY: SET_ACCEL_PWM 직통 (수동 가속)
                                FAULT  : PWM = 0
```

E2E Profile 01 (MTR_CMD, SAF010)

```
Byte 0~3: SET_PWM_LF/RF/LR/RR (int8 × 4)
Byte 4 (low nibble): MTR_RC  (Rolling Counter 4bit)
Byte 5             : MTR_CRC (CRC-8/AUTOSAR, poly 0x2F, init 0xFF)
Arduino 수신측: CRC/RC 검증 실패 → 30ms 타임아웃 → 모터 PWM=0 (safe-state)
```

구현 표준

- **AUTOSAR ASW** (MPC5606B): C99 + MISRA-C:2012 (SWR024) — Simulink/Embedded Coder 자동 생성
- **RPi Python**: 3.13 (uv 관리, Hailo NPU 호환), type hint, dataclass interfaces
- **Arduino C++**: Arduino IDE, DFRobot_MCP2515 + Wire
- **도구**: MATLAB R2024b + AUTOSAR Blockset + mobilgene + S32 Design Studio

적용

Full / CL3 · DBC-driven (RPi) + ARXML-driven (ECU) — 인터페이스 변경이 코드에 자동 반영

3.4 · 03. SWE

↑ 목차로

SWE.4-6 SW 검증·통합·자격 시험

Unit Verification + Integration + Qualification — 통합 운영

핵심

- **SWE.4** 단위 검증 + **SWE.5** 통합 시험 + **SWE.6** 자격 시험을 통합 운영 (Full/CL3)
- SYS.4-5 시연일에 결합 수행 — 동일 환경에서 단위→통합→자격 순차 실행

SWE.4 단위 검증 — 타깃별 도구

타깃	검증 방법	도구	커버리지 목표
RPi Python (acc_*)	pytest 단위 + 타입 체크	pytest, mypy, ruff	구문 80% (안전 critical 100%)
Arduino C++	호스트 사이드 단위 + Serial 시나리오	Cppcheck	주요 함수 통과/실패
MPC5606B AUTOSAR ASW	Simulink Test 하네스 + 정적 분석	Simulink Test, Polyspace/Cppcheck	MC/DC 80% (ASIL B 100%)
Simulink 모델 (FSM, PID)	모델 기반 단위 시험	Simulink Test	Decision Coverage 100%

핵심 단위 시험 케이스

UT ID	SWR	시나리오
UT-FSM-01	SWR001	OFF + ACC ON 토글 + 자가진단 OK → STANDBY (STK022 정정 반영)
UT-FSM-02	SWR002	FOLLOWING + 브레이크 → STANDBY (50ms 이내, SAF004)
UT-MTR-01	SWR034	inner-loop PI: setpoint=50, 측정=0 → 양의 PWM 출력
UT-CAN-01	SWR012	encode_acc_ctrl(BTN_ACC_SET=1) → 4바이트, 비트 위치 검증
UT-CAN-02	SWR015	3주기 미수신 → ERR 플래그 set
UT-FUS-01	SWR004	VEH_DET=1 + LiDAR 5m → VEH_DIST=5000mm 송출
UT-E2E-01	SAF010	잘못된 MTR_CRC 수신 → Arduino motor_set_pwm(0)

SWE.5 통합 시험 — RPi 측 6단계 (시뮬→실 CAN)

1. `acc_can` 단독 — cantools encode/decode 라운드트립
2. `acc_can + acc_hmi` — 시뮬 모드 GUI (`uv run python acc_hmi/main.py`) 로 `[SIM] TX 0x...` 로그 확인
3. `+ acc_fusion` — Camera/LiDAR 더미 → SENSOR_FUSION 송신
4. 실 CAN (vcan0 → MCP2515) — socketcan 가상 → 실 하드웨어
5. Arduino #1 ↔ #2 (I2C) — 모터 응답 확인
6. 풀 RPi+ECU+MTR — SYS.4 Stage 4 와 결합

핵심 통합 시험 케이스

IT ID	SWR	합격 기준
IT-01	SWR012/030	HMI 버튼 → ACC_CTRL → ECU 수신, 50ms 내 BTN 비트 일치
IT-02	SWR018/019	ECU ACC_STATUS → HMI 4 표시 항목 갱신, 100ms 내
IT-03	SWR013/014	FOLLOWING 진입 + 거리 추종 PID 수렴
IT-04	SWR015	SENSOR HB 60ms 끊김 → ERR_SENSOR set + Diag → FAULT
IT-05	SWR021~023	태스크 주기 측정값 ±10% 이내 (10/20/50ms)

SWE.6 자격 시험 — SWC 단위 시나리오

QT ID	SWC	합격 기준
QT-01	AccStateMachine	5상태 전이 매트릭스 전수 PASS, 정의되지 않은 전이 무시
QT-02	DistanceControl	step input 후 settling ≤ 2초, overshoot ≤ 20%
QT-03	MotorControl	inner PI settling ≤ 1초, steady-state error ≤ 5%
QT-04	CanCommunication	9 메시지 주기 ±10%, 타임아웃 동작 확인
QT-05	HmiManager + Diag	FAULT 진입 1초 내 HMI 경고 표시
QT-06	코드 품질	MISRA-C:2012 Mandatory + Required 위반 0건

적용

Full / CL3 · 통합 SYS.4-5 시연일에 단위→통합→자격 결합 수행. CI 자동화는 M4 CDR 전 GitHub Actions 도입 권장.

HW.1 HW 요구사항

HWR 12건 + 실제 부품 인벤토리 (Light)

핵심

- **적용 수준**: Light — 범용 평가보드 사용, ISO 26262 Part 5 가정 명시로 갈음

HW 인벤토리 (실측)

HWR	실제 부품
HWR001 메인 ECU	NXP TRK-MPC5606B 평가보드 (e200z0h, 64MHz, FlexCAN)
HWR002 모터 제어	Arduino Uno × 2 (CAN node + Motor node)
HWR003 인식 보드	Raspberry Pi 5 8GB + Hailo-8L NPU (13 TOPS)
HWR004 LiDAR	Slamtec RPLiDAR (12m 거리)
HWR005 카메라	USB 카메라 또는 Picamera v3
HWR006 CAN 트랜시버	DFRobot CAN BUS Shield (MCP2515 + TJA1040), 종단 120Ω × 2
HWR007 모터	DC 엔코더 모터 × 4 + Arduino Motor Shield R3 × 2 (스택)
HWR008 인코더	모터 통합 인코더 (실제 1개만 신호 연결, 4채널 broadcast)
HWR009 운전자 입력	RPi PyQt5 가상 GUI + 페달 슬라이더
HWR010 HMI	RPi 디스플레이 (HDMI / 터치)
HWR011 차체	1/10 스케일 RC카
HWR012 전원	모터 7~12V 배터리 + 로직 USB 5V

HWR 전체 (12건)

UID	제목
HWR001	메인 ECU - MPC5606B
HWR002	모터 제어 보드 - Arduino (2대 구성)
HWR003	카메라+LiDAR 처리 유닛
HWR004	LiDAR 센서
HWR005	카메라 모듈
HWR008	인코더
HWR006	CAN 트랜시버
HWR007	모터 및 모터 드라이버
HWR009	운전자 입력 장치 (가상 GUI + 페달)
HWR010	HMI 표시 장치 (GUI 겸용)
HWR011	차체 프레임
HWR012	전원 공급

HW.2 HW 설계

결선도 + 핀맵 + 모터 배선 (Light)

핵심 — 결선도

```
RPi5 —USB— Camera, LiDAR
RPi5 —CAN(MCP2515)─┐
                   ├─ 500kbps + 종단 120Ω × 2
ECU MPC5606B —FlexCAN─┤
                   │
Arduino #1 —DFRobot CAN BUS Shield─┘
Arduino #1 —I2C 100k (0x10)— Arduino #2 —Motor Shield × 2— DC Encoder Motor × 4
                                                          └ Encoder D2/D4

전원: 모터 7~12V 배터리 → 위쪽 Motor Shield Vin / 로직 USB 5V (모든 GND 한 점 공통)
```

핀맵 (Arduino #1 — CAN)

핀	역할
D2	CAN INT (미사용 — 폴링 방식)
D10	CAN BUS Shield SPI CS (`CAN_CS_PIN 10`)
D11/12/13	SPI MOSI/MISO/SCK
A4 / A5	I2C SDA / SCL → Arduino #2 직결

핀맵 (Arduino #2 — Motor)

핀	역할
D2 (INT0)	엔코더 A
D4	엔코더 B
D3, D8, D9, D11, D12, D13	Motor Shield R3 (스택 ×2)
A4 / A5	I2C Slave 0x10

모터 배선 (극성 주의)

- Shield 1 A → 앞L (극성 뒤집기), Shield 2 A → 뒤L (극성 뒤집기)
- Shield 1 B → 앞R (정방향), Shield 2 B → 뒤R (정방향)

적용

Light · 정형 회로도(KiCAD) 미작성. 결선 절차서로 갈음. 출처: `ACC_ARDUINO/README.md`

↑ 목차로

HW.3 HW 검증

VT-01~07 검증 단계 (Light)

핵심 — 검증 단계 (보텀업)

단계	대상	합격 기준
VT-01	전원	5V ±0.2V, 모터 전원 유효 범위, GND 공통 도통
VT-02	I2C 단독	NACK 미발생, 3바이트 라운드트립 OK

VT-03	모터 단체	4륜 정/역 회전, 차체 전진 방향 확인 (PWM 80)
VT-04	엔코더 PPR	`DEBUG_ENCODER=1` + 정확히 1바퀴 → 카운트 일정 → `ENCODER_PPR` 기록
VT-05	CAN BUS Shield	PCAN-View 0x210 송신 → 시리얼 로그 OK, 잘못된 ID 무시
VT-06	CAN 종단	Recessive ~2.5V, Dominant Diff ~2V 파형 안정
VT-07	RPi5 + 센서	`acc_fusion/demo/` 스크립트 실행 OK, 12m 까지 거리 측정

적용

Light · 환경 시험(온도/EMC) 없음. 기능 검증만.

HW.4 HW 통합 시험

HW-INT-01~06 (Light)

핵심 — 통합 단계

1. **HW-INT-01** Arduino #2 단독 (모터 + 엔코더) — VT-03/04
2. **HW-INT-02** Arduino #1 단독 (CAN + I2C 마스터) — VT-05
3. **HW-INT-03** #1 ↔ #2 (I2C 통합) — 가상 모터 명령 → 응답
4. **HW-INT-04** + ECU(MPC5606B) CAN 연결 — 풀 CAN bus (3 노드)
5. **HW-INT-05** + RPi5 (CAN + HMI) — 4 노드 풀 통합
6. **HW-INT-06** 전체 차체 탑재 → SYS.4-5 시연 입력

합격 기준

- 모든 노드가 CAN 버스에 연결된 상태에서 24시간 통신 안정 (HB 카운터 정상 증가)
- 모터 4륜 동기 회전 (좌우 균형)
- RPi 디스플레이에 HMI 정상 표시
- 부팅 순서: 모터 OFF → ECU → RPi → HMI → 모터 enable

적용

Light

SUP.1 품질 보증

Automotive SPICE 4.0 — Quality Assurance · Full / CL3

1. 목적

본 문서는 ASPICE SUP.1(Quality Assurance) 및 ISO 26262 Part 2 Clause 6(Confirmation Reviews) 요구사항에 따라, 1/10 스케일 RC 카 ACC 시스템 개발 프로젝트의 품질 보증 활동을 계획하고 관리하기 위한 문서이다.

품질 목표, 프로세스 감사 계획, 산출물 리뷰 기준, 시정 조치 절차, ISO 26262 Confirmation Review 계획을 포함한다.

2. QA 계획 (Quality Assurance Plan)

2.1 품질 목표

품질 목표	측정 지표	목표 기준
요구사항 추적성 완전성	Traceability Matrix 커버리지	STK → SYS → SWR/HWR 100% 연결
ASIL B 요구사항 구현 완성도	ASIL B 항목 시험 통과율	100%
MISRA-C 코딩 규칙 준수	정적 분석 도구 위반 건수	Mandatory / Required 위반 0건
산출물 리뷰 완료율	리뷰 완료된 산출물 비율	마일스톤별 100%
부적합 시정 조치율	CAR 발행 후 기한 내 해결 비율	Critical 24h, High 3일 내 100%
빌드 성공률	CI 빌드 성공 비율	M4 이후 95% 이상

2.2 QA 담당자

역할	담당자	책임
QA 책임자	김유림 (PM)	QA 계획 수립·유지, 감사 주관, CAR 관리, QA 보고서 작성
QA 참여자	엄지혜, 박종범, 유다희	프로세스 준수, 리뷰 참여, NC 발생 시 시정 조치 이행
안전 검토자	김유림 (PM)	ISO 26262 Confirmation Review 수행

2.3 감사 및 리뷰 일정

마일스톤	감사/리뷰 대상	형태
M1 (SRR)	STK / SYS / SAF / MAN.3 / SUP 문서	문서 리뷰
M2 (PDR)	AUTOSAR SWC 설계서, CAN DB, HW 설계서	설계 리뷰
M4 (CDR)	전체 소스코드, Simulink 모델, MISRA-C 정적 분석 결과	코드 리뷰 + 정적 분석
M5	단위·통합·시스템 시험 결과 보고서	시험 결과 리뷰
M6	Safety Case, 전체 문서 최종본	최종 문서 감사
상시	스프린트 완료 산출물	스프린트 리뷰

3. 부적합 사항 분류

등급	정의	처리 기한
Critical	ASIL B 요구사항 미준수, 안전 관련 NC	24시간 이내
Major	주요 프로세스 누락, 추적성 단절	3일 이내
Minor	문서 형식 오류, 경미한 절차 미준수	다음 스프린트 내

6. ISO 연계: Confirmation Review

ISO 26262 Part 2 Clause 6에 따라 ASIL B 대상 안전 산출물은 작성자와 독립된 검토자가 확인 리뷰를 수행. 본 프로젝트에서는 유다희(안전 관리자)가 작성한 안전 산출물을 김유림(안전 검토자)이 독립 검토.

리뷰 대상	근거 조항	시기

HARA 및 Safety Goals (SAF001~003)	ISO 26262-3	M1
FSR — 기능 안전 요구사항(SAF004~008)	ISO 26262-4	M1
TSR — SW 안전 요구사항(SAF010~013)	ISO 26262-6	M2
TSR — HW 안전 요구사항(SAF009, SAF014)	ISO 26262-5	M2
ASIL B SWR 요구사항 할당 결과	ISO 26262-6	M2
ASIL B 시험 결과 보고서	ISO 26262-4	M5
Safety Case	ISO 26262-2	M6

7. 2026-04-30 QA 감사 결과

7.1 식별된 NC 요약

NC ID	등급	발견 경위	부적합 내용	관련 산출물
NC-2026-001	**Critical**	자동 정합성 분석	Arduino acc_can_node MTR_CMD 페이로드 포맷이 옛 6바이트(checksum) 주석 기준 — 현재 DBC E2E P01 (4×PWM + MTR_RC + MTR_CRC, SAF010 ASIL-B) 와 미스매치	ACC_ARDUINO/acc_can_node/can_handler.h/.cpp
NC-2026-002	**Major**	자동 정합성 분석	CAN ID 머지 후 ID 미반영 (4 파일)	_dbc.py, ACC-RPI/CLAUDE.md, Arduino config.h, Arduino README.md
NC-2026-003	**Major**	추적성 분석	SYS.2 ↔ STK / SWE.1 양방향 트레이스 미보강 (RELATIONS Parent 부재)	ACC-docs/ASPICE/SYS2.sdoc
NC-2026-004	**Major**	추적성 분석	SWE.1 산출물 미작성 — 코드 ↔ SYS.2 추적 단절	(SWE1.sdoc 부재)
NC-2026-005	**Major**	정합성 분석	STK019 (9종 입력 버튼) ↔ DBC ACC_CTRL (4 BTN) 매핑 룰 미명시	STK.sdoc, SYS2.sdoc, Project.dbc
NC-2026-006	**Minor**	HW 분석	1-엔코더로 4채널 MTR_SPD_FB broadcast 정책 비명문화	ACC_ARDUINO/acc_motor_node/config.h

7.2 CAR 등록부

CAR 번호	NC	등급	담당	시정 조치	완료 기한	상태
CAR-2026-001	NC-2026-001	Critical	Arduino 담당 (ACC-33/34)	can_handler.cpp 본문이 새 E2E P01 포맷에 맞춰 구현되었는지 검토 → 미구현 시 재작성 + 회귀 테스트. SUP.10 CR-SAF 동시 생성	**2026-04-30** (24h)	OPEN
CAR-2026-002	NC-2026-002	Major	CAN 담당 (ACC-34)	_dbc.py 주석, CLAUDE.md, Arduino config.h, README 갱신. SUP.10 CR-DOC + CR-SW	2026-05-02 (3일)	OPEN
CAR-2026-003	NC-2026-003	Major	요구사항 (ACC-18)	SYS2.sdoc 의 각 REQUIREMENT 에 RELATIONS Parent → STKxxx 추가. SWE.1 작성 후 Child 보강	2026-05-02	OPEN
CAR-2026-004	NC-2026-004	Major	요구사항 (ACC-19)	SWE1.sdoc 초안 작성 — 모듈 단위(acc_fusion / acc_can / acc_hmi / acc_can_node / acc_motor_node) SWR 발급 + SYS.2 트레이스	2026-05-02	OPEN
CAR-2026-005	NC-2026-005	Major	요구사항 + HMI/CAN	SYS.2 IF-001 또는 신규 절에 STK019 ↔ ACC_CTRL 매핑 표 추가 (속도± → SET_ACC_SPD, 거리1/2/3 → SET_ACC_LVL=1/2/3, 일시정지 → BTN_ACC_CANCEL)	2026-05-02	OPEN
CAR-2026-006	NC-2026-006	Minor	HW + 요구사항	STK 또는 SYS.2 에 "1-엔코더로 4채널 동일값 broadcast" 명문화 + 4엔코더 확장은 backlog 명시	다음 스프린트	OPEN

7.3 ISO 26262 Confirmation Review 영향

CAR-2026-001 (E2E P01 미스매치) 은 SAF010 (TSR-02) 와 직접 연관 — **M2 PDR 전 재리뷰 필요** 항목으로 등재.

7.4 후속 조치 (QA 책임자 → 김유림)

1. 위 6건 CAR 를 Jira 에 등록 (CAR-2026-001 ~ CAR-2026-006)
2. CAR-001 은 SUP.10 CR-SAF 와 동시 진행 + 전원 합의 회의 일정
3. 매주 월요일 본 등록부 갱신 → MAN.6 § 5.4 결함 밀도 메트릭과 동기화
4. CAR Closed 기준: 코드 수정 + 회귀 테스트 통과 + QA(김유림) 서명

SUP.8 형상 관리

Automotive SPICE 4.0 — Configuration Management · Full / CL3

항목	내용
프로젝트	1/10 스케일카 Adaptive Cruise Control (ACC) 시스템
문서 ID	ACC-SUP8-CMP-001
버전	1.1
작성일	2026-04-06 (1.0), 2026-04-30 (1.1 — § 10 추가)
적용 표준	ASPICE 4.0 SUP.8, ISO 26262 Part 8 Clause 7
ASIL 등급	ASIL-B

1. 목적

본 문서는 ACC 프로젝트의 형상 관리 활동에 대한 계획을 정의한다. 프로젝트 전 생명주기에 걸쳐 형상 항목의 식별, 버전 관리, 베이스라인 관리, 변경 통제 절차를 수립하여 산출물의 무결성과 추적성을 보장한다. ISO 26262 Part 8 Clause 7에 따라 안전 관련 형상 항목에 대한 추가 통제 규칙도 함께 정의한다.

3. 형상 항목 식별

3.1 산출물 관리 정책

구분	관리 도구	대상	근거
추적성 필요 산출물	StrictDoc + GitHub	요구사항(STK/SYS/SWR/HWR/SAF), ASPICE V-모델 추적 체인	ASPICE 양방향 추적성, ISO 26262 안전 추적성
프로세스/관리 산출물	Confluence	SUP/MAN 산출물, 회의록	협업/편집/Jira 연동 중심
코드/모델/빌드	GitHub	소스코드, Simulink 모델, ARXML, 빌드 산출물	Git 버전 관리

3.2 관리 대상 (요구사항)

분류	형상 항목	저장 위치
이해관계자 요구사항	STK.sdoc	GitHub (reqs/)
시스템 요구사항	SYS.sdoc	GitHub (reqs/STK/)
SW 요구사항	SWR.sdoc	GitHub (reqs/STK/SYS/)
HW 요구사항	HWR.sdoc	GitHub (reqs/STK/SYS/)
안전 요구사항	SAF.sdoc	GitHub (reqs/STK/SYS/)

3.4 형상 항목 식별자 규칙

ACC-[분류]-[항목약어]-[순번] 형식. 분류: SYS / SWE / ARC / MBD / TST / SUP / SAF.

4. 버전 관리

4.1 Git 브랜치 전략

```
main ──────────────────────────────────── 릴리즈 (태그)
  └── dev ──────────────────────────────── 통합 브랜치
        ├── feat/[이슈번호]-[설명] ──────── 기능 개발
        └── fix/[이슈번호]-[설명] ──────── 버그 수정
```

브랜치	용도	보호 규칙
main	릴리즈 가능 안정 버전	PR 필수, 최소 1명 리뷰 승인
dev	개발 통합	PR 필수
feat/*	기능 개발	—
fix/*	버그 수정	—

4.2 태깅 규칙 (Semantic Versioning)

v[MAJOR].[MINOR].[PATCH] — 예: v0.1.0 (초기), v1.0.0 (최종 릴리즈)

4.3 커밋 메시지 (Conventional Commits)

`<type>(<scope>): <description>` — type: feat / fix / docs / refactor / test / chore

6. 변경 통제 (안전 관련 추가 절차)

ASIL-B 형상 항목 변경:

추가 통제 항목	내용
안전 영향 분석	E2E, WDG, 이중 GPIO 등 안전 메커니즘 영향 평가
승인 권한	2명 이상의 리뷰 승인 필요
추적성 확인	안전 요구사항 ↔ 설계 ↔ 코드 ↔ 테스트
회귀 테스트	안전 관련 테스트 케이스 전체 재실행

안전 관련 형상 항목: AccStateMachine SWC, DistanceControl SWC, E2E 통신 보호 (MTR_CMD MTR_RC/MTR_CRC), Watchdog, 브레이크 이중 GPIO, 프로그램 흐름 모니터링, 안전 critical CAN (VEH_CTRL 0x120 / MTR_CMD 0x210 / ECU_HEARTBEAT 0x480).

10. 2026-04-30 실제 형상 항목 인벤토리

10.1 워크스페이스 디렉토리 구조 (실측)

```
/Users/yurim/Desktop/ACC/
├── ACC-CANDB/         (git repo)  - DBC 정본
├── ACC-RPI/           (git repo)  - Raspberry Pi 측 Python 코드
├── ACC-docs/          (git repo)  - 요구사항(.sdoc) + 프로세스 문서
├── ACC_ARDUINO/       (git repo)  - Arduino 펌웨어 (Uno×2)
├── ACC_Diagnostics.slx           - Simulink 진단 모델
└── ACC_Types.sldd                - Simulink Data Dictionary
```

10.2 ACC-CANDB 형상 항목 (정본)

파일	역할	비고
acc_db.dbc	ACC 단독 DBC, 9 메시지	RPi cantools 가 import 시점 로드
Project.dbc	acc_db + Project-original 머지본	mobilgene/CANoe 통합 시 사용 (정본)
Project-original.dbc	mobilgene 교육용 베이스	**수정 금지** — 비교/감사용

M2 PDR 베이스라인 기준:

ID	메시지	Tx	Rx	DLC	주기	ASIL/E2E
0x110	SENSOR_FUSION	SENSOR	ECU	3	20ms	QM
0x111	SENSOR_HEARTBEAT	SENSOR	ECU	2	20ms	QM
0x120	VEH_CTRL	SENSOR	ECU	2	20ms	**ASIL-B** (SAF004/015)
0x210	MTR_CMD	ECU	MTR	6	10ms	**ASIL-B + E2E P01** (SAF010)
0x310	MTR_SPD_FB	MTR	ECU/SENSOR	8	10ms	QM (이전 0x300)
0x320	MTR_HEARTBEAT	MTR	ECU	2	10ms	QM (이전 0x310)
0x480	ECU_HEARTBEAT	ECU	SENSOR/MTR	2	10ms	**ASIL-B** (SAF018, 이전 0x410)
0x510	ACC_CTRL	SENSOR	ECU	4	50ms	QM
0x520	ACC_STATUS	ECU	SENSOR	4	50ms	QM

10.6 마일스톤별 베이스라인 후보

베이스라인	시점	포함 형상 항목	태그 후보
BL-M1 (SRR)	STK + SYS1 동결	STK.sdoc, SYS1.sdoc, MAN/SUP 문서 v1	acc-docs/v0.1.0
BL-M2 (PDR)	SYS2 + DBC + 설계 동결	SYS2.sdoc, acc_db.dbc, Project.dbc, ACC_Diagnostics.slx	acc-docs/v0.2.0 + ACC-CANDB/v0.2.0

BL-M4 (CDR)	SWE1 + 코드 동결	SWE1.sdoc, ACC-RPI/, *ACC_ARDUINO/*	ACC-RPI/v1.0.0 + ACC_ARDUINO/v1.0.0
BL-M5	통합 시험 동결	위 + 시험 결과 보고서	release tag per repo
BL-M6	최종 동결	Safety Case + 위 전부	v1.0.0 (모든 repo)

10.7 형상 감사 후속 조치

- § 10.1 누락 (MPC5606B / STM32 코드 repo) 확인 필요
- § 10.2 ID 시프트는 후행 CR-DES 등록 (SUP.10 § 1.1)
- 형상 식별자 규칙(ACC-[분류]-[항목약어]-[순번]) 이 실제 GitHub repo 명명과 일치 여부 정리 필요

SUP.9 문제 해결 관리

Automotive SPICE 4.0 — Problem Resolution Management · Full / CL3

1. 목적

ACC 프로젝트 수행 중 발생하는 모든 결함·불일치를 식별하고, Jira·Git·StrictDoc 세 도구를 연계하여 해결 과정 전체를 추적·기록하기 위한 최소 절차를 정의한다. 이슈 수정이 요구사항·설계 변경을 수반하는 경우 SUP.10 변경 요청(CR) 절차와 연계하여 처리한다.

2. 도구별 역할 분담

도구	역할	SUP.9에서 하는 일
Jira	이슈 등록 · 상태 관리	이슈 생성, 담당자 배정, 상태 전환(OPEN→CLOSED), 원인 댓글
Git	수정 이력 보존	커밋 메시지에 이슈 번호 포함, PR로 수정 검증
StrictDoc	요구사항 변경 이력 보존	버그로 인해 요구사항이 바뀔 때만 사용

3. 이슈 분류 기준

구분	등급	정의	처리 기한
Severity	S1	안전 기능 침해 또는 시스템 동작 불가	24시간
Severity	S2	주요 기능 오작동	3일 이내
Severity	S3	경미한 결함·개선	마감(4/30)
Priority	P1	프로젝트 차단 또는 안전 관련	24시간
Priority	P2	다음 통합 전까지 필수	3일 이내
Priority	P3	마감 전 처리 권고	마감(4/30)

4. 이슈 처리 절차

Step 1. 이슈 등록 — Jira (필수: 제목, 발견일/발견자, Severity/Priority, 관련 요구사항 ID, 담당자)
Step 2. 수정 — Git (CR 연계 시 feat/CR-NNN, 단순 버그 fix/fix-ACC-NNN. 커밋 메시지 [ACC-NNN] fix: 수정 내용 (요구사항ID))
Step 3. 요구사항 변경 시 — StrictDoc (sdoc 업데이트 + COMMENT에 변경 이유/이슈 번호)
Step 4. 검증 및 종료 — Jira

상태	정의	전환 조건
OPEN	등록 완료, 미착수	등록 즉시
IN PROGRESS	수정 진행 중	Git 브랜치 생성 후
RESOLVED	수정 완료, 검증 대기	Git PR Merge 완료 후
CLOSED	검증 완료	S1: 시나리오 미재현 + 전원 합의 / S2·S3: 담당자 동작 확인
WONT FIX	팀 합의로 미수정	S3 한정 (ASIL-B 이슈는 금지)

5. 추적 및 에스컬레이션

일일 추적: 매일 스탠드업(10분)에서 Jira OPEN/IN PROGRESS 현황 공유. P1 미해결 시 팀 채팅 즉시 알림.

조건	조치
P1 이슈 24h 내 IN PROGRESS 미전환	팀 채팅 즉시 알림 → 담당자 재지정
P1 이슈 수정 후 48h 내 CLOSED 미전환	팀 전체 즉시 회의
OPEN 이슈 5개 이상 누적 (마감 1주 전)	팀 회의 → 우선순위 재조정
S1 이슈 수정이 요구사항·설계 변경 수반	SUP.10 CR-SAF/CR-REQ 즉시 생성
ASIL-B 요구사항 변경 발생	SUP.10 CR-SAF + StrictDoc 업데이트 + 전원 합의 필수

7. SUP.10 연계 흐름

상황	SUP.9 처리	SUP.10 연계	StrictDoc
단순 코드 버그 수정	Jira → Git → CLOSED	불필요	불필요
버그 수정 중 요구사항 수치 변경	Jira 유지	CR-REQ 생성	필수
버그 수정 중 설계(CAN/SWC) 변경	Jira 유지	CR-DES 생성	불필요
ASIL-B 항목 관련 버그 수정	S1/P1 자동 승격 + 전원 합의	CR-SAF	필수

8. 2026-04-30 식별 이슈 등록부

8.1 이슈 등록부 (Jira 후보)

임시 ID	제목	Severity	Priority	발견일	관련 요구사항 ID	CR 연계	담당 (Jira)	상태
SUP9-2026-001	Arduino MTR_CMD 페이로드 포맷이 옛 6바이트(checksum) 주석 — 현재 DBC E2E P01 (4×PWM + MTR_RC + MTR_CRC) 와 미스매치	**S1**	**P1**	2026-04-30	SAF010, SYS2-SR-002 (TSR-02)	**CR-SAF**	ACC-33/34 (Arduino)	OPEN
SUP9-2026-002	CAN ID 머지 후 (MTR_SPD_FB 0x310, MTR_HEARTBEAT 0x320, ECU_HEARTBEAT 0x480) 4 파일에 옛 ID 잔존	S2	P2	2026-04-30	SYS2-IF-002	**CR-DOC** + **CR-SW**	ACC-34 (CAN)	OPEN
SUP9-2026-003	STK019 (9종 입력 버튼) ↔ DBC ACC_CTRL (4 BTN) 매핑 룰 부재 — HMI 인코딩 모호	S2	P2	2026-04-30	STK019, SYS2-IF-001	**CR-REQ**	ACC-31 + ACC-18	OPEN
SUP9-2026-004	SWE.1 산출물 부재로 코드 ↔ SYS.2 트레이스 단절 (ASPICE NC)	S2	P2	2026-04-30	(SYS2 → SWE1 미연결)	(ACC-19 직접)	ACC-19	OPEN
SUP9-2026-005	1-엔코더로 4채널 MTR_SPD_FB broadcast 정책 비명문화	S3	P3	2026-04-30	STK 신설 필요	**CR-REQ**	ACC-33	OPEN
SUP9-2026-006	L/R 어댑터 변환 (4ch → 2pair) 미합의	S3	P3	2026-04-30	SYS2-IF-001 (확장)	**CR-DES**	ACC-34	OPEN

8.3 에스컬레이션

- **001 (S1/P1)**: 24h 내 IN PROGRESS 전환 안 되면 팀 채팅 즉시 알림 + ASIL-B CR-SAF 동시 + 전원 합의 회의 → **2026-04-30 처리**
- **002~003 (S2/P2)**: 3일 내 RESOLVED 전환 → 2026-05-02
- **004**: SWE.1 작성은 ACC-19 본 작업으로 직접 추진
- **005~006 (S3/P3)**: 다음 스프린트 내 처리 또는 합의 후 WONT FIX

8.4 추적성 완성 체크 (§6 조건)

- [ ] Jira ACC-NNN 등록 → 본 표의 임시 ID 매핑 갱신
- [ ] Git 커밋 메시지에 [ACC-NNN] fix: ... 형식 적용
- [ ] PR 설명에 Jira 링크 포함
- [ ] 001/003/005 의 경우 StrictDoc 업데이트 (sdoc 의 COMMENT 에 ACC-NNN + CR-NNN 기재)
- [ ] 001 의 경우 SUP.10 CR-SAF 와 양방향 링크

SUP.10 변경 요청 관리

Automotive SPICE 4.0 — Change Request Management · Full / CL3

1. 변경 요청 절차

모든 변경은 아래 6단계 절차를 통해 처리된다.

단계	활동	수행 주체	산출물
1	**CR 제출**	변경 요청자	Jira CR 티켓 (CR-NNN)
2	**영향 분석**	담당 ECU 개발자	영향 분석 결과 (Jira 코멘트), ASIL 영향 분석서 (CR-SAF)
3	**CCB 검토/승인**	전체 팀원 합의	승인/반려 결정 (Jira), Confluence 회의록
4	**구현**	담당 개발자	Git PR (dev/CR-NNN), StrictDoc 업데이트
5	**검증/재검증**	테스트 담당자	테스트 결과 보고서, 회귀 테스트 통과
6	**CR 종료**	PM	Closed CR 티켓, Git 태그 (v-CR-NNN)

1.1 CR 유형 분류 및 관리 도구

CR 유형	변경 범위	관리 도구	승인 절차
CR-SAF 안전 변경	ASIL 항목, Fail-safe, HMI 경고	StrictDoc + Git	전원 합의 + ISO 26262 안전 영향 분석
CR-REQ 요구사항 변경	기능 요구사항, ICD	StrictDoc + Jira	전원 합의
CR-DES 설계 변경	아키텍처, CAN, PID, AUTOSAR SWC	Git + Jira + Confluence	전원 합의
CR-SW 코드 변경	ECU 소스코드, AUTOSAR SWC	Git (PR)	담당자 + 1인 리뷰
CR-DOC 문서 변경	설계 문서, 테스트 케이스, 회의록	Jira + Confluence	해당 담당자 처리

3. CCB 승인 기준

소규모 팀(4~6인) 구성으로 별도 CCB 조직 미구성, CR 유형별 합의 범위 운영.

CR 유형	검토 구성원	승인 절차	긴급 변경
CR-SAF	전체 팀원 전원	Confluence 회의 / Jira 전원 승인 + ISO 26262 안전 영향 분석. 반대 1건이라도 보류	PM 즉시 결정 → 당일 전원 보고
CR-REQ	전체 팀원 전원	Jira 전원 Approve + StrictDoc 변경 이력. 합의 기한 2일	PM 결정 후 팀 공유
CR-DES	전체 팀원 전원	Confluence 검토 / Jira Approve. 합의 기한 2일	담당자 결정 후 팀 공유
CR-SW	담당자 + 리뷰어 1인	Git PR Merge. 안전 관련 변경은 CR-SAF 격상	담당자 즉시 처리
CR-DOC	해당 담당자	Confluence 업데이트 후 통보	담당자 즉시 처리

6. ISO 26262 Part 8 Clause 7 연계

SUP.10 요구사항	ISO 26262 조항	ACC 적용
변경 요청 관리 절차	Part 8 § 7.4.1	CR 유형 분류 + Jira 추적
영향 분석	Part 8 § 7.4.2	CR-SAF 시 영향 분석 체크리스트 + ASIL 판단
변경 후 검증	Part 8 § 7.4.3	CR 유형별 재검증 범위. CR-SAF 안전 시나리오 전수 재검증
ASIL 항목 변경 시 안전 영향 분석	Part 8 § 7.4.4	Fail-safe / HMI 경고 변경 시 안전 영향 분석서 + 전원 합의
형상 관리 연계	Part 8 § 7.4.5	Git 태그 + StrictDoc 변경 이력

8. 2026-04-30 신규 CR 등록부

8.1 CR 등록부

임시 CR ID	CR 유형	제목	영향 범위	ASIL 영향	CCB 승인	연계 SUP
CR-2026-001	**CR-SAF**	Arduino MTR_CMD 처리: 옛 6바이트 → 신 E2E P01	ACC_ARDUINO/acc_can_node/can_handler.cpp/.h, .ino	**있음** (SAF010 ASIL-B, TSR-02)	전원 합의 + 안전 영향	SUP9-2 / ACC-3

		(4×PWM + MTR_RC + MTR_CRC) 적용			분석 첨부	
CR-2026-002	**CR-DOC**	CAN ID 머지 후 옛 ID 잔존 문서 갱신: _dbc.py 주석, ACC-RPI/CLAUDE.md, ACC_ARDUINO/README.md	위 3개 파일	없음	담당자 처리	SUP9-2 / ACC-3
CR-2026-003	**CR-SW**	Arduino acc_can_node/config.h CAN ID 값 갱신 (MTR_SPD_FB 0x300→0x310, MTR_HEARTBEAT 0x310→0x320, ECU_HEARTBEAT 0x410→0x480)	config.h + can_handler.cpp 호출처	간접 (ECU_HEARTBEAT = SAF018)	담당자 + 리뷰어 1인 → CR-SAF 격상 검토	SUP9-2 / ACC-3
CR-2026-004	**CR-REQ**	STK019 ↔ DBC ACC_CTRL 매핑 룰 SYS.2 IF 신설 (속도± → SET_ACC_SPD, 거리1/2/3 → SET_ACC_LVL=1/2/3, 일시정지 → BTN_ACC_CANCEL)	STK.sdoc, SYS2.sdoc, acc_db.dbc	검토 필요	전원 합의 (2일)	SUP9-2 / ACC-3 ACC-18
CR-2026-005	**CR-REQ**	"1-엔코더로 4채널 MTR_SPD_FB 동일값 broadcast" STK 신규 또는 SYS.2 NF 명문화 + 4엔코더 backlog 이관	STK.sdoc, SYS2.sdoc	없음	전원 합의 (2일)	SUP9-2
CR-2026-006	**CR-DES**	L/R 어댑터 (4ch MTR_CMD → 2pair PWM) 변환 룰 SYS.2 IF-001 명문화 (LF=LR, RF=RR 가정)	SYS2.sdoc IF-001, acc_can_node 주석	없음	전원 합의 (2일)	SUP9-2 / ACC-3
CR-2026-007	**CR-DES** (후행)	CAN DB 머지 시점의 ID 시프트 후행 등록 — 정식 CR 절차 미경유로 SUP.8 §10.2 형상 추적 갭 보전	acc_db.dbc, Project.dbc	있음 (간접)	사후 전원 보고 + Confluence 회의록	— (감사

8.3 처리 우선순위 (긴급도)

1. **CR-2026-001 (CR-SAF, 24h)** — 즉시 PM 결정 + 당일 전원 보고 (긴급 변경 절차)
2. **CR-2026-003 (CR-SW → CR-SAF 격상 검토)** — CR-001 와 연계 처리
3. **CR-2026-002 (CR-DOC)** — 1일 내 담당자 처리
4. **CR-2026-004, 005, 006 (CR-REQ/CR-DES)** — 2일 내 전원 합의
5. **CR-2026-007 (사후 추적)** — 다음 베이스라인 갱신 시 일괄 처리

8.4 ISO 26262 Part 8 Clause 7 적용

CR-2026-001, 003, 007 은 안전 관련(직접/간접):
- 변경 요청 문서화 ✓ (본 등록부)
- 영향 분석 ✓
- 변경 후 검증 — Git PR + 회귀 테스트
- ASIL 항목 변경 시 안전 영향 분석 — **CR-001 발행 시 별도 안전 영향 분석서 첨부 필수**
- 형상 관리 연계 — Git 태그 + StrictDoc + SUP.8 §10 인벤토리 갱신

VAL.1 밸리데이션

Automotive SPICE 4.0 — Validation · Partial / CL2

1. 프로세스 목적 (Purpose)

시스템이 의도된 사용 환경에서 이해관계자의 니즈를 충족함을 확인한다.

2. Base Practices

BP	활동
VAL.1.BP1	밸리데이션 전략 수립
VAL.1.BP2	밸리데이션 환경 구축
VAL.1.BP3	시나리오 정의
VAL.1.BP4	밸리데이션 실행
VAL.1.BP5	결과 분석·보고
VAL.1.BP6	Safety Case 작성

3. 산출물 (Output Work Products)

WP ID	설명	본 프로젝트 위치
08-90	Validation Strategy	Confluence: VAL.1
08-92	Scenario Catalog	SIL 시나리오 카탈로그
13-50	Validation Report	M5/M6 결과 보고서
13-50	Safety Case	ISO 26262 Part 2 Safety Case (D 작성)

4. 테일러링 적용

Partial / CL2 — SIL 시나리오 + 간이 HIL + Safety Case

5. 본 프로젝트 진행 현황

- 시나리오: 정상 주행 (CRUISING/FOLLOWING) · 운전자 오버라이드 (브레이크/액셀 50ms) · FAULT 진입/복귀 · SET·RES 재활성화
- 실차 RC카 시범 주행: 저속·비공공도로·직접 감독 환경 (1/10 스케일)
- Safety Case: HARA→SG→FSC→TSR→SwSafety→검증 전 체인 증거 종합 + 잔여 리스크 평가

6. 추적성

입력 (Upstream)

SYS.5

출력 (Downstream)

프로젝트 종료

요구사항 추적 매트릭스 (RTM)

STK → SYS → SWR → SAF → HWR → 검증

요구사항 추적 매트릭스 (Requirement Traceability Matrix)

STK → SYS → SWR → SAF → HWR → 검증 항목 → Test Case 양방향 트레이스. 본 표는 자동 생성된 스냅샷이며 정본은 StrictDoc `strictdoc export` 결과.

STK ↔ SYS.1 (이해관계자)

STK.sdoc 의 RELATIONS Parent 가 SYS1-STK-xxx 를 가리키도록 100% 연결됨.

STK	제목	Parent (SYS1-STK-···)
STK001	시스템 목적 (1/10 RC카 ACC)	SYS1-STK-001/003/013/014
STK002	전방 차량 인식 (RPi 카메라)	SYS1-STK-004
STK003	차간거리 측정 (LiDAR)	SYS1-STK-004
STK004~006	속도/순항/거리 단계	SYS1-STK-001/014
STK007~008	운전자 오버라이드 (브레이크/액셀)	SYS1-STK-001/005/011
STK009	5상태 머신	SYS1-STK-001/005
STK019~021	HMI 입력/표시/그레이아웃	SYS1-STK-001/010/013
STK022	ACC ON 동작 흐름	SYS1-STK-005/012
STK024	최소 설정 속도 5cm/s	SYS1-STK-001

STK ↔ SYS.2 (시스템 요구사항)

> **NC-2026-003:** SYS2.sdoc 의 RELATIONS Parent → STK 미보강. CAR-2026-003 진행 중.

SYS.2 ID	카테고리	예상 STK Parent
SYS2-FR-001	인식 기능	STK002, STK003
SYS2-FR-002	속도 제어	STK004, STK005
SYS2-FR-003	FOLLOWING / 안전거리	STK003, STK006
SYS2-FR-004	오버라이드	STK007, STK008
SYS2-FR-005	HMI 입력/표시	STK019, STK020, STK021
SYS2-FR-006	상태 머신	STK009, STK022
SYS2-PR-001	응답 시간 / 주기	STK007 (50ms)
SYS2-SR-001	FSR 요약	STK007, STK008, STK009
SYS2-SR-002	TSR 요약 (E2E P01)	SAF010
SYS2-IF-001	센서 데이터 포맷	STK002, STK003
SYS2-IF-002	CAN 통신 매트릭스	STK009, STK020

SYS.2 ↔ SWE.1 (SW 요구사항)

> **NC-2026-004:** SWE.1.sdoc 미작성. 본 표는 코드 모듈 기준 예상 매핑.

SYS.2	예상 SWR	구현 모듈
FR-001 (인식)	SWR-FUS-001~	ACC-RPI/acc_fusion/

FR-005 (HMI)	SWR-HMI-001~	ACC-RPI/acc_hmi/
FR-006 (FSM)	SWR-ECU-FSM-001~	ACC_Diagnostics.slx (FSM)
SR-002 (E2E P01)	SWR-ARD-CAN-001~	ACC_ARDUINO/acc_can_node/
IF-002 (CAN)	SWR-CAN-001~	ACC-RPI/acc_can/, ACC_ARDUINO/acc_can_node/

SYS ↔ HWR (HW 요구사항)

SYS.2	HW 매핑
FR-001 (인식)	RPi5 + Camera + RPLiDAR + Hailo NPU
FR-002~003 (제어)	NXP TRK-MPC5606B (FlexCAN)
FR-003 (모터 구동)	Arduino Uno + Motor Shield R3 ×2 + DC 엔코더 ×4
IF-002 (CAN)	DFRobot CAN BUS Shield (MCP2515) + 120Ω 종단

안전 체인 (ISO 26262)

HARA → Safety Goal → FSR → TSR → SW Safety Req → 구현 → 검증

ISO 26262 안전 체인

HARA → Safety Goal → FSC (FSR) → TSC (TSR) → SW Safety Requirement → 구현 → 검증.

HARA (Hazard Analysis & Risk Assessment)

Hazard ID	시나리오	S	E	C	ASIL	대응 SG
H-01	의도치 않은 가속 (Unintended Acceleration)	S2	E3	C3	ASIL C	SG-01
H-02	브레이크 미작동 (Loss of Deceleration)	S3	E4	C3	ASIL D	SG-02
H-03	의도치 않은 급감속 (Unintended Deceleration)	S2	E3	C2	ASIL B	SG-03

Safety Goals → FSR → TSR (요약)

SG	안전 목표	FSR	TSR	ASIL
SG-01	의도치 않은 가속을 방지	FSR-01: 액셀 입력 시 50ms 내 ACC OFF/STANDBY	TSR-01: 액셀 페달 입력 latency < 30ms	ASIL C
SG-02	브레이크 미작동 방지	FSR-02: 브레이크 입력 시 50ms 내 STANDBY + 모터 0	TSR-02 (SAF010): MTR_CMD E2E Profile 01 (RC + CRC8) 적용, 30ms 타임아웃 시 safe-state	ASIL D
SG-03	의도치 않은 급감속 방지	FSR-03: 잘못된 거리 측정에 의한 자동 감속 방지	TSR-03: SENSOR_FUSION 무결성 — 60ms 미수신 또는 ERR_SENSOR≠0 → FAULT	ASIL B

안전 critical CAN 메시지

ID	메시지	ASIL	안전 메커니즘	관련 TSR
0x120	VEH_CTRL (브레이크/액셀)	ASIL B	20ms 주기 + ERR_SENSOR 플래그	FSR-01/02
0x210	MTR_CMD (4륜 PWM)	ASIL B	E2E P01 (MTR_RC + MTR_CRC), 30ms 타임아웃 → 모터 0	TSR-02 (SAF010)
0x480	ECU_HEARTBEAT	ASIL B	10ms 주기, 3주기(30ms) 미수신 → MTR safe-state	SAF018

SW Safety Requirements ↔ 구현 매핑

TSR / SW Safety Req	구현 위치	검증 항목
TSR-02 (E2E P01)	ACC_ARDUINO/acc_can_node/can_handler.cpp + ECU AUTOSAR ASW	잘못된 RC/CRC 송신 시 모터 0 응답 확인
SAF018 (HB 30ms)	Arduino loop() + ECU 송신 주기	ECU 정지 → 30ms 내 모터 0
SAF004 (브레이크 50ms)	SENSOR (RPi) 페달 → CAN VEH_CTRL → ECU FSM	End-to-end 응답 ≤ 50ms

안전 활동 마일스톤

마일스톤	안전 활동
M1 SRR	Item Definition + HARA + Safety Goals 확정 + Confirmation Review (SAF001~003)
M2 PDR	FSR / TSR 확정 + ASIL 할당 + Confirmation Review (SAF004~014)
M4 CDR	SW Safety Req → 코드 트레이스 확립 + 정적 분석 결과
M5	Fail-safe 시나리오 전수 검증 + Confirmation Review
M6	Safety Case 작성 + 최종 Confirmation Review